



Universidad Tecnológica Nacional

Tecnicatura Universitaria en Programación

Arquitectura y Sistemas Operativos

TRABAJO PRÁCTICO INTEGRADOR

Seguridad de la Información

***Sistema Automatizado de Detección y Respuesta ante Intentos de
Fuerza Bruta en un Servidor Linux***

Autor :

Mateo Fernandez

Fecha de Entrega

13/06/2026



ÍNDICE

1. Fase 1 – Análisis del Problema
2. Fase 2 — Diseño de la Arquitectura del Sistema
3. Fase 3 – Implementación del Laboratorio
4. Fase 4 – Implementación de la Solución
5. Fase 5 - Planeación del ataque
6. Fase 6 - Validación Experimental
7. Fase 7 - Análisis de Evidencias y Correlación de Eventos
8. Conclusiones
9. Bibliografía

Fase 1 – Análisis del Problema

1. Contexto empresarial

Para el desarrollo del presente Trabajo Práctico Integrador se plantea un escenario basado en una organización ficticia denominada **MAT Solutions S.A.**, dedicada al desarrollo de software y a la administración de infraestructura informática para clientes nacionales e internacionales. Como parte de sus operaciones diarias, la empresa dispone de un servidor Linux que aloja diferentes servicios internos y permite la administración remota mediante el protocolo Secure Shell (SSH).

Debido a la naturaleza distribuida de las tareas de administración, el acceso remoto constituye un recurso indispensable para el personal técnico. Sin embargo, la exposición permanente del servicio SSH a la red incrementa la superficie de ataque del sistema, convirtiéndolo en un objetivo frecuente para intentos de acceso no autorizados mediante técnicas automatizadas de fuerza bruta.

En este contexto, la organización requiere implementar mecanismos que permitan supervisar continuamente los intentos de autenticación, registrar los eventos relevantes y responder de manera automática ante comportamientos considerados como maliciosos, fortaleciendo así la seguridad del servidor y reduciendo el riesgo de comprometer información crítica.

2. Activo crítico



El activo principal que se busca proteger es el servidor Linux utilizado por la organización para la administración remota y la prestación de servicios internos. Particularmente, el servicio SSH representa un componente crítico de la infraestructura, ya que constituye el mecanismo utilizado por los administradores para gestionar el sistema operativo de forma remota.

La confidencialidad, integridad y disponibilidad de este servicio resultan fundamentales para garantizar el correcto funcionamiento de la infraestructura informática, por lo que cualquier intento de acceso no autorizado representa un riesgo significativo para la organización.

3. Amenaza identificada

La principal amenaza considerada en este trabajo corresponde a los ataques de fuerza bruta dirigidos contra el servicio SSH.

Este tipo de ataque consiste en la ejecución automatizada de múltiples intentos de autenticación utilizando diferentes combinaciones de nombres de usuario y contraseñas, con el objetivo de descubrir credenciales válidas que permitan acceder al sistema.

Actualmente, estas técnicas pueden ejecutarse mediante herramientas especializadas capaces de realizar cientos o miles de intentos de autenticación en muy poco tiempo, incrementando considerablemente la probabilidad de comprometer cuentas que utilicen contraseñas débiles, o predecibles.

Aunque el protocolo SSH incorpora mecanismos criptográficos robustos para proteger la comunicación, la utilización de credenciales vulnerables (passwords *comúnmente*

conocidas, estándar, o utilizadas por default por diversos equipos informáticos) continúa siendo uno de los vectores de ataque más frecuentes sobre este servicio.

4. Vulnerabilidades detectadas

Durante el análisis del escenario se identificaron diversas condiciones que incrementan el riesgo de sufrir un ataque exitoso de fuerza bruta. Entre ellas pueden mencionarse:

- Utilización de contraseñas débiles o fácilmente predecibles.
- Exposición permanente del servicio SSH a la red.
- Ausencia de mecanismos automáticos de detección de intentos reiterados de autenticación fallida (patrón de comportamiento malicioso) .
- Falta de políticas de bloqueo temporal de direcciones IP sospechosas.
- Insuficiente monitoreo de los archivos de registro del sistema.
- Ausencia de mecanismos automatizados de respuesta ante incidentes de seguridad.

Estas condiciones, combinadas, incrementan significativamente la probabilidad de que un atacante logre obtener acceso no autorizado al sistema.

5. Impacto potencial

La explotación exitosa de esta amenaza podría generar consecuencias de diversa gravedad para la organización.

Entre los principales impactos se encuentran el acceso no autorizado a información confidencial, la modificación o eliminación de archivos críticos, la instalación de software

malicioso, la elevación de privilegios por parte del atacante, la interrupción parcial o total de los servicios ofrecidos por el servidor y el eventual compromiso completo de la infraestructura informática.

Además del impacto técnico, este tipo de incidentes puede ocasionar pérdidas económicas, afectación de la continuidad operativa y daños reputacionales para la organización.

6. Propuesta de solución

Con el objetivo de mitigar los riesgos identificados, se propone el diseño e implementación de un **Sistema Automatizado de Detección y Respuesta ante Intentos de Fuerza Bruta en un Servidor Linux**.

La solución estará orientada a supervisar permanentemente los intentos de autenticación realizados sobre el servicio SSH, registrar todos los eventos relevantes en los archivos de auditoría del sistema, detectar patrones compatibles con ataques de fuerza bruta y ejecutar respuestas automáticas destinadas a impedir la continuidad del ataque.

Para ello se utilizarán diferentes componentes propios del sistema operativo Linux, tales como OpenSSH, rsyslog, auditd, Fail2Ban y el firewall del sistema, permitiendo demostrar cómo interactúan entre sí para implementar un mecanismo integral de protección basado en la monitorización continua y la respuesta automatizada.

7. Objetivo general

Diseñar, implementar y validar un sistema automatizado capaz de detectar, registrar eventos y responder ante intentos de fuerza bruta dirigidos contra el servicio SSH de un servidor Linux, utilizando herramientas nativas del sistema operativo y mecanismos automáticos de protección.

8. Objetivos específicos

- Configurar un servidor Linux con acceso remoto mediante SSH.
 - Implementar mecanismos de registro y auditoría de eventos relacionados con la autenticación.
 - Simular un ataque de fuerza bruta mediante herramientas especializadas en un entorno de laboratorio controlado.
 - Detectar automáticamente los intentos reiterados de autenticación fallida.
 - Implementar el bloqueo automático de direcciones IP consideradas maliciosas.
 - Validar el correcto funcionamiento de la solución mediante pruebas experimentales y evidencias obtenidas durante la ejecución del caso práctico.
 - Analizar el funcionamiento conjunto de los distintos componentes del sistema operativo involucrados en la detección y respuesta ante incidentes de seguridad.
-

9. Alcance del proyecto

El presente trabajo se desarrollará íntegramente dentro de un entorno virtualizado y controlado, conformado por una máquina atacante y una máquina víctima, sin afectar sistemas reales ni redes hogareñas o corporativas.

Las pruebas realizadas tendrán exclusivamente fines académicos y permitirán evaluar el comportamiento del sistema operativo frente a intentos controlados de acceso no autorizado, demostrando la eficacia de los mecanismos implementados para la detección y mitigación de este tipo de amenazas.

Fase 2 — Diseño de la Arquitectura del Sistema

Diseño de la Infraestructura

Con el propósito de desarrollar un entorno de pruebas seguro y completamente aislado, se implementará un laboratorio virtual compuesto por dos máquinas virtuales interconectadas mediante una red privada. Esta arquitectura permitirá simular un escenario real de ataque sin comprometer equipos o redes de producción.

La primera máquina virtual representará al atacante y ejecutará herramientas especializadas para la realización de ataques de fuerza bruta sobre el servicio SSH. La segunda máquina virtual representará el servidor de la organización, sobre el cual se implementarán los mecanismos de monitoreo, auditoría y respuesta automática.



La comunicación entre ambas máquinas se realizará exclusivamente a través de una red interna virtual, garantizando que todas las pruebas permanezcan confinadas al entorno de laboratorio.

La arquitectura propuesta se compone de los siguientes elementos:

- Máquina Atacante (Kali Linux)

- Simulación del atacante.
- Ejecución de Hydra para ataques de fuerza bruta.
- Ejecución de Nmap para tareas de reconocimiento.
- Verificación de la conectividad con el servidor.

- Máquina Servidora (Ubuntu Server)

- Servicio OpenSSH para acceso remoto.
- Sistema de registro de eventos mediante rsyslog y journald.
- Auditoría de eventos utilizando auditd.
- Detección automática de ataques mediante Fail2Ban.
- Aplicación de reglas de bloqueo mediante el firewall del sistema.

Esta arquitectura permite representar de manera simplificada un escenario de infraestructura empresarial donde un servidor Linux debe responder automáticamente frente a intentos reiterados de autenticación no autorizada.

Especificaciones del Laboratorio Virtual

Característica	Máquina Atacante	Máquina Servidora
Sistema Operativo	Kali Linux	Ubuntu Server LTS
Función	Simulación del atacante	Servidor protegido
CPU	2 núcleos	2 núcleos
Memoria RAM	2 GB	2 GB
Disco	20 GB	20 GB
Adaptador de Red	Red Interna Virtual	Red Interna Virtual
Herramientas Principales	Hydra, Nmap	OpenSSH, Fail2Ban, auditd, rsyslog



Diseño de la Red

Para garantizar un entorno de pruebas completamente controlado se utilizará una red privada virtual con direccionamiento IPv4 perteneciente al segmento 192.168.100.0/24.

La configuración prevista será la siguiente:

- Red: 192.168.132.0/24
- Máscara: 255.255.255.0

Máquina Atacante

- Dirección IP: 192.168.132.129

Servidor Linux

- Dirección IP: 192.168.132.128

La utilización de una red privada permitirá aislar completamente el laboratorio del resto de la infraestructura física, evitando interferencias externas y garantizando la seguridad durante la ejecución de las pruebas experimentales.

Rol de cada componente

Componente	Función dentro del proyecto
OpenSSH	Recibe y procesa las solicitudes de autenticación remota.
sshd	Demonio encargado de gestionar las conexiones SSH.
systemd	Inicializa y administra los servicios del sistema operativo.
journald	Registra eventos generados por los servicios del sistema.

<code>rsyslog</code>	Centraliza y almacena los registros en archivos del sistema.
<code>auth.log</code>	Contiene los eventos relacionados con la autenticación de usuarios.
<code>auditd</code>	Realiza auditorías detalladas de eventos de seguridad.
<code>Fail2Ban</code>	Analiza los registros e identifica patrones compatibles con ataques de fuerza bruta.
Firewall (<code>iptables</code> o <code>nftables</code>)	Bloquea automáticamente la dirección IP del atacante.

Arquitectura Funcional del Sistema

El funcionamiento del sistema desarrollado se basa en la cooperación de diversos componentes propios del sistema operativo Linux, cada uno de los cuales desempeña una función específica dentro del proceso de detección y respuesta frente a intentos de acceso no autorizados.

A diferencia de una solución basada en una única aplicación, la propuesta aprovecha distintos servicios nativos del sistema operativo que trabajan de forma coordinada para monitorear continuamente la actividad del servidor, registrar los eventos de seguridad y ejecutar respuestas automáticas cuando se detectan patrones compatibles con un ataque de fuerza bruta.

A continuación se describe el rol de cada uno de los componentes involucrados.

OpenSSH

OpenSSH constituye la implementación libre del protocolo Secure Shell (SSH), ampliamente utilizado para la administración remota segura de sistemas Linux y Unix.



Su principal función consiste en establecer comunicaciones cifradas entre un cliente y un servidor, garantizando la confidencialidad e integridad de la información intercambiada durante la sesión.

Dentro del presente proyecto, OpenSSH representa el punto de ingreso al sistema, ya que todas las solicitudes de autenticación realizadas por usuarios legítimos o atacantes son recibidas inicialmente por este servicio.

Cada intento de autenticación es procesado por el servidor SSH, el cual verifica la existencia del usuario, valida las credenciales proporcionadas y determina si el acceso debe ser autorizado o rechazado.

Cuando una autenticación falla, OpenSSH registra inmediatamente el evento para que posteriormente pueda ser analizado por otros componentes del sistema operativo encargados del monitoreo y la seguridad.

Demonio sshd

El servicio OpenSSH es administrado por el proceso denominado **sshd** (Secure Shell Daemon), el cual se ejecuta continuamente en segundo plano esperando solicitudes de conexión provenientes de la red.

En los sistemas Linux, un demonio es un proceso que permanece activo sin requerir interacción directa del usuario, ofreciendo servicios permanentes al sistema operativo.

Cuando un cliente intenta establecer una conexión SSH, el demonio sshd recibe la solicitud, crea los procesos necesarios para gestionar la sesión y realiza el procedimiento de autenticación correspondiente.

systemd



Systemd constituye el sistema de inicialización y administración de servicios utilizado por la mayoría de las distribuciones Linux modernas.

Su función principal consiste en iniciar, detener, supervisar y reiniciar automáticamente los distintos servicios del sistema operativo, asegurando su correcto funcionamiento durante todo el ciclo de vida del sistema.

Dentro del presente proyecto, systemd será responsable de mantener operativos servicios fundamentales como OpenSSH, Fail2Ban, rsyslog y auditd.

Además, proporciona mecanismos para consultar el estado de cada servicio, visualizar registros y verificar que todos los componentes encargados de la seguridad permanezcan correctamente ejecutándose durante las pruebas experimentales.

`systemd-journald`

Systemd-journald es el componente encargado de recopilar y almacenar los eventos generados por los distintos servicios administrados por systemd.

Cada vez que un servicio produce información relevante, como el inicio de una sesión, un error de autenticación o una advertencia del sistema, journald registra dicho evento en su base de datos de registros.

Posteriormente, estos registros pueden ser consultados mediante herramientas de administración, permitiendo reconstruir cronológicamente las actividades realizadas dentro del sistema operativo.

En el presente proyecto, journald constituye una de las primeras fuentes de información utilizadas para verificar el comportamiento del servidor durante la ejecución del ataque simulado.



rsyslog

Rsyslog es el servicio encargado de centralizar, procesar y almacenar los mensajes generados por el sistema operativo y sus diferentes aplicaciones.

Su principal función consiste en organizar los eventos recibidos y escribirlos en archivos de registro específicos, facilitando posteriormente las tareas de monitoreo, auditoría y análisis de incidentes.

Dentro del presente proyecto, rsyslog registrará los eventos de autenticación generados por OpenSSH en el archivo **/var/log/auth.log**, permitiendo conservar un historial completo de todos los intentos de acceso realizados sobre el servidor.

Estos registros constituirán la principal fuente de información utilizada por Fail2Ban para detectar patrones compatibles con ataques de fuerza bruta.

Archivo auth.log

El archivo **/var/log/auth.log** constituye uno de los registros de seguridad más importantes del sistema operativo Linux.

En él se almacenan cronológicamente todos los eventos relacionados con los procesos de autenticación, incluyendo inicios de sesión exitosos, intentos fallidos, utilización de privilegios administrativos y otras actividades vinculadas con el acceso al sistema.

Durante la ejecución del caso práctico, este archivo permitirá observar en tiempo real cada uno de los intentos de autenticación generados por la herramienta Hydra, convirtiéndose en la principal evidencia del ataque de fuerza bruta desarrollado en el laboratorio.

Asimismo, este registro será analizado automáticamente por Fail2Ban para determinar cuándo una dirección IP supera el umbral permitido de intentos fallidos.



Además de administrar las conexiones remotas, sshd genera los eventos relacionados con cada intento de acceso, proporcionando información que posteriormente será utilizada por los mecanismos de auditoría y detección implementados en el sistema.

Sistema Automatizado de Detección y Respuesta

Una vez que los intentos de autenticación han sido registrados por los mecanismos de logging del sistema operativo, comienza la segunda etapa del proceso de protección: la detección y respuesta automática.

En esta fase intervienen diversos componentes especializados cuya función consiste en analizar continuamente los registros generados por el sistema, identificar comportamientos anómalos y ejecutar acciones preventivas destinadas a impedir que un ataque continúe desarrollándose.

A diferencia de un mecanismo de monitoreo pasivo, el sistema implementado posee la capacidad de actuar de forma autónoma ante eventos previamente definidos, reduciendo considerablemente el tiempo de respuesta frente a incidentes de seguridad.

Fail2Ban

Fail2Ban es una herramienta de seguridad diseñada para proteger servicios expuestos a la red frente a ataques automatizados de fuerza bruta.

Su funcionamiento se basa en el monitoreo continuo de archivos de registro generados por distintos servicios del sistema operativo. Mediante el uso de expresiones regulares y reglas configurables, analiza los eventos registrados e identifica patrones compatibles con intentos reiterados de autenticación fallida.



Cuando una dirección IP supera el número máximo de intentos permitido dentro de un intervalo de tiempo determinado, Fail2Ban interpreta dicho comportamiento como potencialmente malicioso y ejecuta automáticamente una acción de mitigación.

En el presente proyecto, Fail2Ban supervisará el archivo **/var/log/auth.log**, detectando los intentos fallidos registrados por OpenSSH y ordenando el bloqueo temporal de la dirección IP atacante mediante el firewall del sistema.

Gracias a este mecanismo, el servidor es capaz de responder automáticamente sin requerir intervención del administrador, reduciendo significativamente la probabilidad de que un ataque de fuerza bruta logre comprometer las credenciales de acceso.

auditd

Auditd constituye el servicio de auditoría del sistema operativo Linux y forma parte del subsistema Linux Audit Framework.

Su principal función consiste en registrar eventos de seguridad de alto nivel relacionados con el acceso a recursos del sistema, modificaciones sobre archivos críticos, utilización de privilegios administrativos y ejecución de determinadas operaciones previamente definidas por el administrador.

En el contexto del presente trabajo, auditd será utilizado como mecanismo complementario de auditoría, permitiendo generar registros adicionales sobre eventos relevantes ocurridos durante la ejecución del laboratorio.

De esta manera, además de los registros producidos por OpenSSH y Fail2Ban, el sistema dispondrá de una fuente adicional de información para reconstruir cronológicamente los acontecimientos y facilitar el análisis posterior del incidente de seguridad.



Firewall del Sistema

El firewall constituye el componente responsable de controlar el tráfico de red autorizado hacia el servidor.

En sistemas Linux modernos esta función puede implementarse mediante *iptables* o *nftables*, dependiendo de la distribución y versión del sistema operativo utilizada.

Dentro del presente proyecto, el firewall no tomará decisiones por sí mismo. En cambio, actuará como mecanismo ejecutor de las acciones ordenadas por Fail2Ban.

Cuando se detecte una dirección IP que haya superado el umbral permitido de intentos fallidos de autenticación, Fail2Ban incorporará dinámicamente una regla al firewall para bloquear toda comunicación proveniente de dicha dirección.

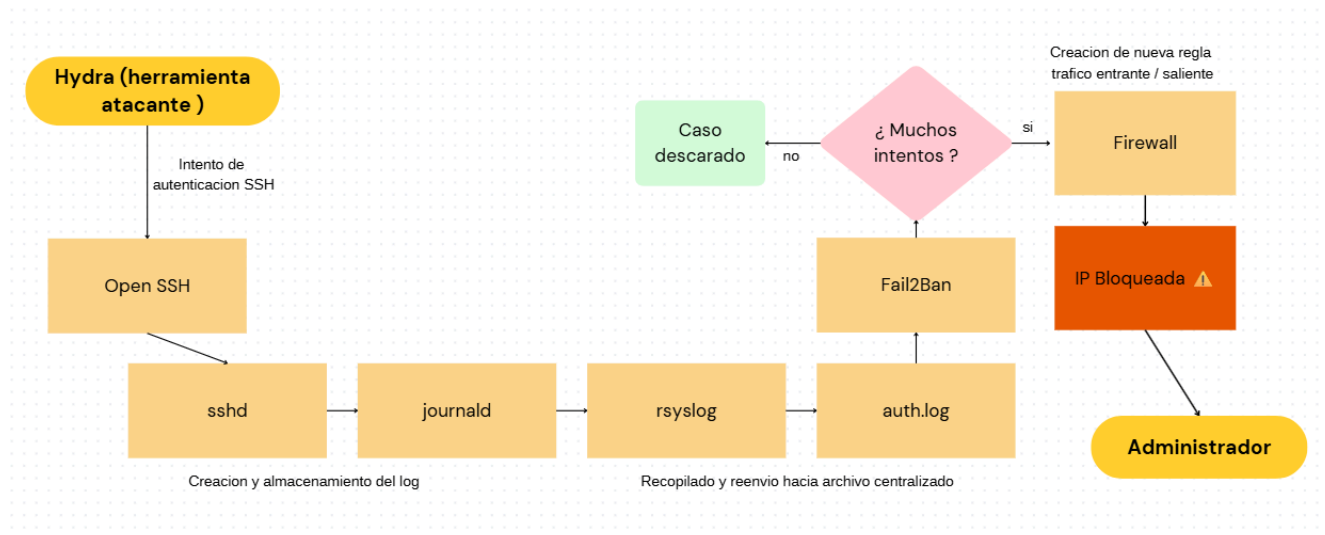
Este procedimiento permite detener automáticamente el ataque sin interrumpir el funcionamiento normal del servicio SSH para el resto de los usuarios autorizados.

Rol del Administrador

Si bien el sistema implementado posee mecanismos automáticos de detección y respuesta, el administrador continúa desempeñando un papel fundamental dentro del proceso de gestión de incidentes.

Una vez producido el bloqueo automático del atacante, el administrador podrá consultar los diferentes registros generados por el sistema para verificar la naturaleza del incidente, identificar la dirección IP involucrada, analizar la cronología de los eventos y validar el correcto funcionamiento de los mecanismos de protección implementados.

Esta capacidad de auditoría resulta indispensable para documentar incidentes de seguridad, realizar investigaciones posteriores y adoptar medidas preventivas adicionales cuando sea necesario.



Fase 3 – Implementación del Laboratorio

Preparación del Entorno de Trabajo

Una vez definida la arquitectura general del sistema, se procedió a la preparación del entorno de laboratorio sobre el cual se desarrollarían las pruebas experimentales.

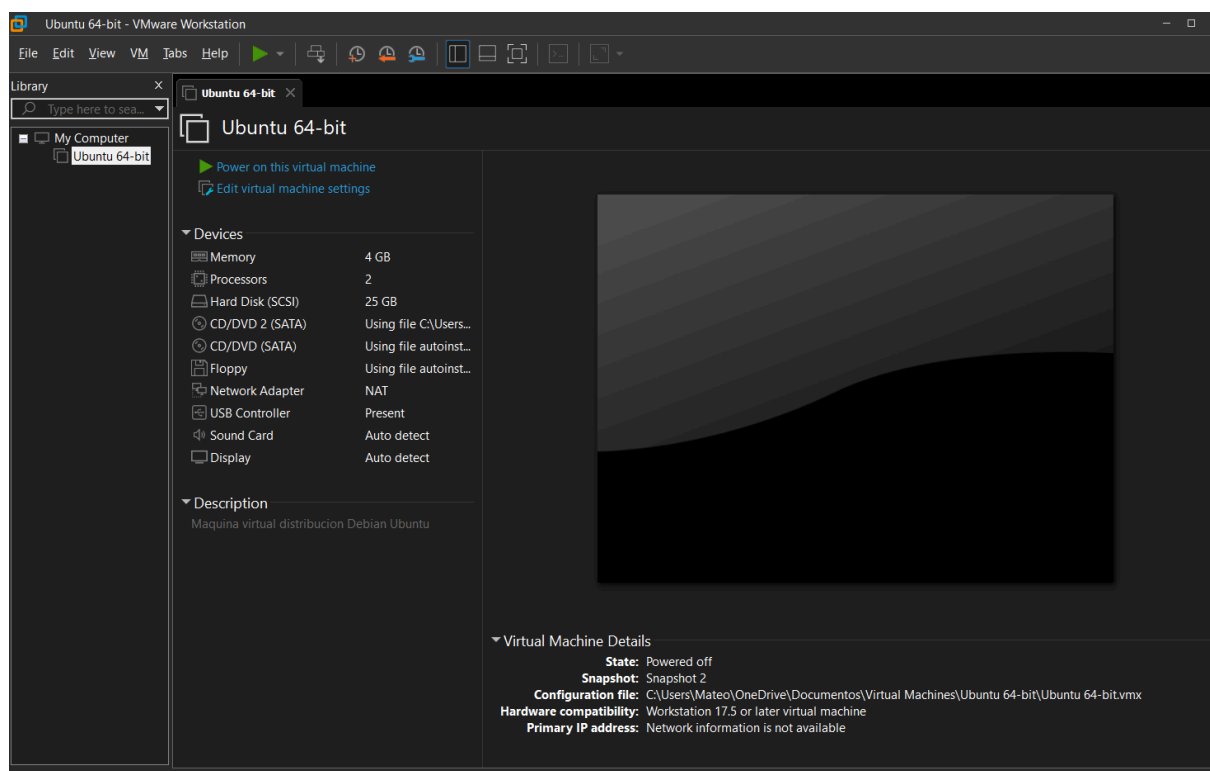
Para la virtualización de los equipos se seleccionó **VMware Workstation**, un hipervisor de tipo 2 ampliamente utilizado tanto en entornos académicos como profesionales para la creación y administración de máquinas virtuales.

Como servidor víctima se reutilizó una máquina virtual previamente instalada con **Ubuntu Desktop 26.04 LTS**, la cual fue destinada a cumplir el rol de servidor Linux expuesto al

servicio SSH. La elección de esta distribución permitió disponer de un entorno gráfico para facilitar las tareas de administración y la obtención de evidencias, sin alterar el funcionamiento de los servicios propios del sistema operativo.

Durante esta etapa también se adoptó una metodología basada en la creación de **Snapshots** o instantáneas de la máquina virtual antes de realizar modificaciones relevantes sobre la configuración del sistema. Esta práctica permite restaurar rápidamente el laboratorio ante posibles errores durante la implementación y constituye una técnica ampliamente utilizada en tareas de administración y pruebas sobre infraestructuras virtualizadas.

Figura 1. Máquina virtual Ubuntu configurada como servidor del laboratorio.



Actualización del Sistema Operativo

Antes de comenzar la instalación de los componentes que formarían parte de la solución propuesta, se realizó una actualización completa del sistema operativo con el objetivo de garantizar un entorno estable, seguro y compatible con las versiones más recientes de los paquetes utilizados durante el proyecto.

Para ello se ejecutaron los siguientes comandos:

```
sudo apt update
```

```
sudo apt upgrade -y
```

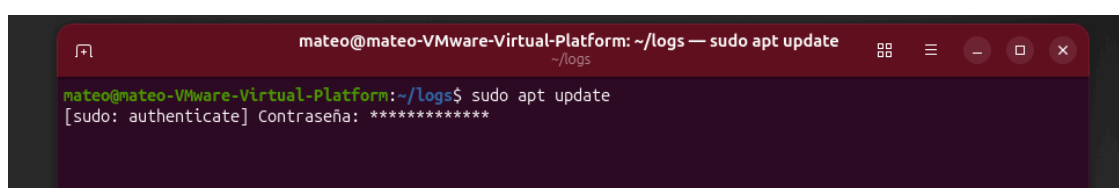
```
sudo apt autoremove -y
```

```
sudo apt autoclean
```

Estas operaciones permitieron sincronizar la base de datos de paquetes del sistema, instalar las actualizaciones disponibles, eliminar dependencias que ya no eran necesarias y liberar espacio ocupado por archivos temporales de instalación.

La actualización previa del sistema constituye una buena práctica de administración, ya que reduce la posibilidad de incompatibilidades entre paquetes, incorpora correcciones de seguridad y asegura que los servicios instalados posteriormente funcionen sobre un entorno correctamente mantenido.

Figura 2. Actualización de paquetes mediante el gestor APT.



```

mateo@mateo-VMware-Virtual-Platform:~/logs$ sudo apt upgrade -y
Get more security updates through Ubuntu Pro with 'esm-apps' enabled:
  libopenexr-3.1-30 libavcodec62 libavfilter11 libavutil60 libswscale9
  python3-pip libswresample6 libavformat62
Learn more about Ubuntu Pro at https://ubuntu.com/pro
Actualizando:
  bpftool                libnfs14                linux-headers-generic-hwe-26.04  perl
  build-essential        libpackagekit-glib2-18  linux-image-generic-hwe-26.04  perl-base
  cups-filters            libperl5.40             linux-libc-dev                  perl-modules-5.40
  cups-filters-core-drivers  libxml2-16              linux-perf                      tar
  gir1.2-packagekitglib-1.0  linux-generic-hwe-24.04  linux-tools-common
  gstreamer1.0-packagekit  linux-generic-hwe-26.04  packagekit

Instalando dependencias:
  libcrypt-dev                linux-image-7.0.0-27-generic        linux-tools-7.0.0-27
  linux-headers-7.0.0-27      linux-main-modules-zfs-7.0.0-27-generic  linux-tools-7.0.0-27-generic
  linux-headers-7.0.0-27-generic  linux-modules-7.0.0-27-generic

Paquetes sugeridos:
  linux-tools

Resumen:
  Actualizando: 22, Instalando 8, Eliminando: 0, no actualizando: 0
  15 standard LTS security updates
  Tamaño de la descarga: 230 MB
  Espacio necesario: 373 MB / 12,6 GB disponible

done
mateo@mateo-VMware-Virtual-Platform:~/logs$ sudo apt autoremove -y
Resumen:
  Actualizando: 0, Instalando 0, Eliminando: 0, no actualizando: 0

mateo@mateo-VMware-Virtual-Platform:~/logs$ sudo apt autoclean

```

Identificación del Servidor

Como parte del proceso de preparación del laboratorio se modificó el nombre del equipo utilizando la herramienta **hostnamectl**, asignándole el identificador **server-linux**.

Esta modificación facilita la identificación del servidor durante las tareas de administración y permite representar de manera más realista el escenario empresarial planteado para el presente trabajo. El cambio fue realizado mediante el siguiente comando:

```
sudo hostnamectl set-hostname server-linux
```

Posteriormente se verificó la correcta aplicación de la configuración consultando nuevamente la información del sistema.

Figura 3. Verificación del nuevo nombre del servidor mediante hostnamectl.

```

mateo@mateo-VMware-Virtual-Platform:~/logs$ hostnamectl
  Static hostname: server-linux
    Icon name: computer-vm
    Chassis: vm
  Chassis Asset Tag: No Asset Tag
    Machine ID: ce3fc1f7352540f690a777ca678593b7
    Boot ID: 65f67645827446ec8399789b83c8b4d4
    AF_VSOCK CID: 2740970585
  Virtualization: vmware
  Operating System: Ubuntu 26.04 LTS
    Kernel: Linux 7.0.0-22-generic
  Architecture: x86_64
  Hardware Vendor: VMware, Inc.
  Hardware Model: VMware Virtual Platform
  Hardware Version: None
  Firmware Version: 6.00
    Firmware Date: Thu 2020-11-12
    Firmware Age: 5y 7month 2w 1d
mateo@mateo-VMware-Virtual-Platform:~/logs$

```

Relevamiento del Sistema Operativo

Una vez configurado el servidor, se realizó un relevamiento de sus principales características técnicas mediante distintas utilidades propias del sistema Linux.

Se utilizaron los comandos:

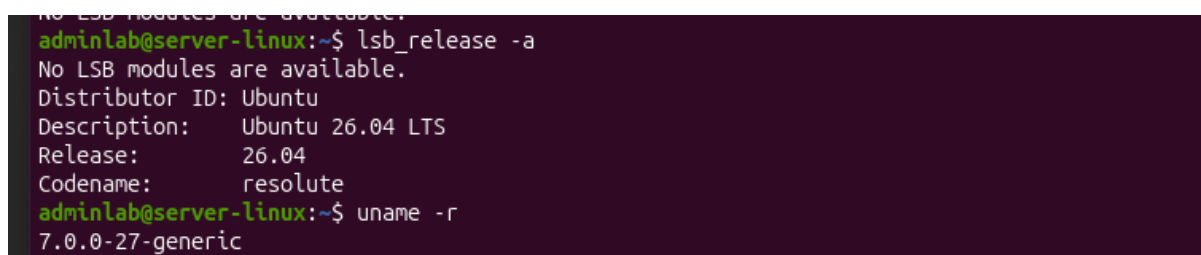
```
hostnamectl  
lsb_release -a  
uname -r  
ip a
```

La información obtenida permitió identificar la versión del sistema operativo, la versión del kernel, la arquitectura del procesador, el tipo de virtualización empleado y la configuración inicial de la interfaz de red.

Durante esta etapa se comprobó que el laboratorio utilizaba **Ubuntu 26.04 LTS**, ejecutándose sobre una máquina virtual VMware con arquitectura **x86_64** y una interfaz de red configurada mediante NAT para permitir el acceso temporal a Internet durante la fase de instalación de software.

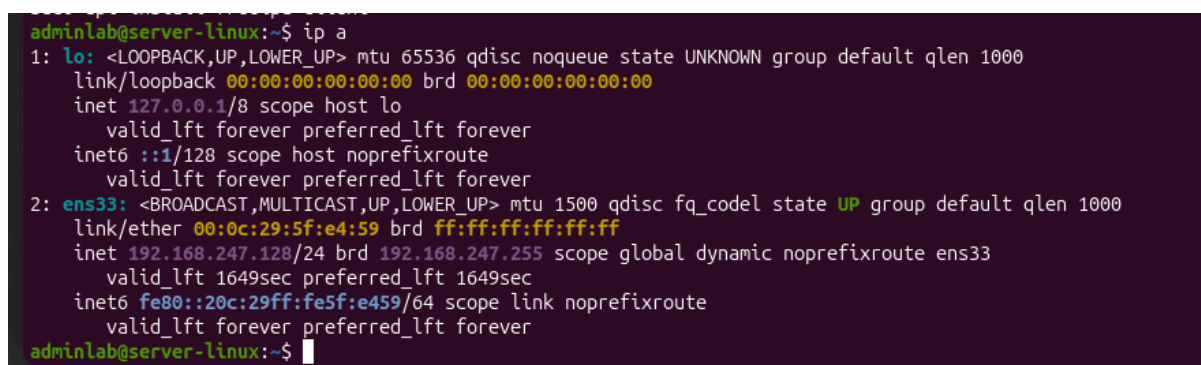
Esta información resulta de utilidad tanto para documentar el entorno experimental como para garantizar la reproducibilidad del laboratorio en futuras implementaciones.

Figura 5. Información de la distribución Linux obtenida mediante lsb_release y versión del Kernel instalada.



```
adminlab@server-linux:~$ lsb_release -a  
No LSB modules are available.  
Distributor ID: Ubuntu  
Description:    Ubuntu 26.04 LTS  
Release:        26.04  
Codename:       resolute  
adminlab@server-linux:~$ uname -r  
7.0.0-27-generic
```

Figura 6. Configuración inicial de la interfaz de red.



```
adminlab@server-linux:~$ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 00:0c:29:5f:e4:59 brd ff:ff:ff:ff:ff:ff  
    inet 192.168.247.128/24 brd 192.168.247.255 scope global dynamic noprefixroute ens33  
        valid_lft 1649sec preferred_lft 1649sec  
    inet6 fe80::20c:29ff:fe5f:e459/64 scope link noprefixroute  
        valid_lft forever preferred_lft forever  
adminlab@server-linux:~$
```

Creación del Usuario Administrador

Con el objetivo de separar las tareas de administración del laboratorio respecto del usuario utilizado originalmente durante la instalación del sistema operativo, se creó una nueva cuenta denominada **adminlab**, destinada exclusivamente a la administración del servidor durante el desarrollo del proyecto.

Posteriormente, dicho usuario fue incorporado al grupo **sudo**, otorgando permisos administrativos para instalar software, modificar configuraciones del sistema y administrar los diferentes servicios involucrados en el laboratorio.

La creación de un usuario específico para la administración constituye una buena práctica de seguridad, ya que permite limitar el uso de cuentas personales y mantener una mejor organización de los privilegios dentro del sistema.

Los comandos utilizados fueron los siguientes:

```
sudo adduser adminlab  
sudo usermod -aG sudo adminlab
```

Finalmente, se verificó el correcto funcionamiento de la cuenta comprobando que el usuario pudiera ejecutar comandos con privilegios administrativos.

Figura 8. Creación del usuario administrador del laboratorio.

```
mateo@mateo-VMware-Virtual-Platform:~/logs$ sudo adduser adminlab
Nueva contraseña:
Vuelva a escribir la nueva contraseña:
passwd: contraseña actualizada correctamente
Cambiando la información de usuario para adminlab
Introduzca el nuevo valor, o pulse INTRO para usar el valor predeterminado
Nombre completo []:
Número de habitación []:
Teléfono del trabajo []:
Teléfono de casa []:
Otro []:
¿Es correcta la información? [S/n] s
mateo@mateo-VMware-Virtual-Platform:~/logs$ sudo usermod -aG sudo adminlab
mateo@mateo-VMware-Virtual-Platform:~/logs$
```

Al finalizar esta etapa, el servidor quedó completamente preparado para comenzar la instalación de los servicios que conformarán el Sistema Automatizado de Detección y Respuesta ante Intentos de Fuerza Bruta, iniciando en la siguiente fase con la implementación del servicio OpenSSH, componente central del escenario de seguridad desarrollado.

Fase 4 – Implementación de la Solución

Instalación del Servicio OpenSSH

Una vez finalizada la preparación del laboratorio, se procedió a instalar el primer componente de la arquitectura propuesta: el servicio **OpenSSH Server**.

La elección de este servicio responde a que constituye el principal mecanismo de administración remota en sistemas Linux y, al mismo tiempo, uno de los servicios más frecuentemente expuestos a Internet dentro de infraestructuras empresariales.

Debido a su amplia utilización, el servicio SSH representa un objetivo habitual para ataques automatizados de fuerza bruta, motivo por el cual fue seleccionado como componente principal del escenario de seguridad desarrollado en el presente trabajo.

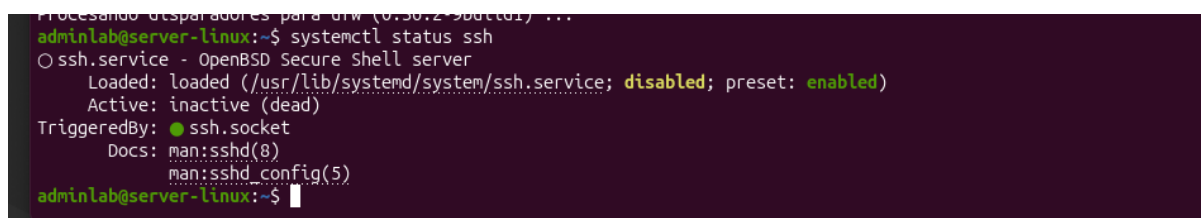
La instalación del servicio se realizó utilizando el gestor de paquetes APT mediante los siguientes comandos:

```
sudo apt update
sudo apt install openssh-server -y
```

Una vez finalizada la instalación, Ubuntu incorporó automáticamente el demonio **sshd**, encargado de permanecer en ejecución escuchando permanentemente nuevas solicitudes de conexión remota.

Este demonio constituye uno de los procesos centrales de la arquitectura implementada, ya que será el primer componente del sistema operativo en recibir los intentos de autenticación generados posteriormente desde la máquina atacante.

Figura 10. Instalación del paquete OpenSSH Server.



```
adminlab@server-linux:~$ systemctl status ssh
○ ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: enabled)
   Active: inactive (dead)
   TriggeredBy: ● ssh.socket
   Docs: man:sshd(8)
         man:sshd_config(5)
adminlab@server-linux:~$
```

Verificación del Servicio

Finalizada la instalación, se verificó el correcto funcionamiento del servicio utilizando las herramientas de administración proporcionadas por **systemd**.

Para ello se ejecutó el siguiente comando:

```
sudo systemctl status ssh
```

La salida obtenida confirmó que el servicio se encontraba en estado **active (running)**, indicando que el demonio **sshd** había sido correctamente iniciado por el sistema de inicialización y administración de servicios.

Posteriormente se configuró el servicio para iniciar automáticamente junto con el sistema operativo mediante el comando:

```
sudo systemctl enable ssh
```

Esta configuración garantiza la disponibilidad permanente del servicio incluso después de reinicios del servidor, reproduciendo el comportamiento esperado en un entorno de producción.

A continuación se verificó que el proceso se encontrara escuchando conexiones sobre el puerto TCP 22 mediante:

```
sudo ss -tulnp | grep :22
```

Finalmente, se consultó la versión instalada del software utilizando:

```
ssh -V
```

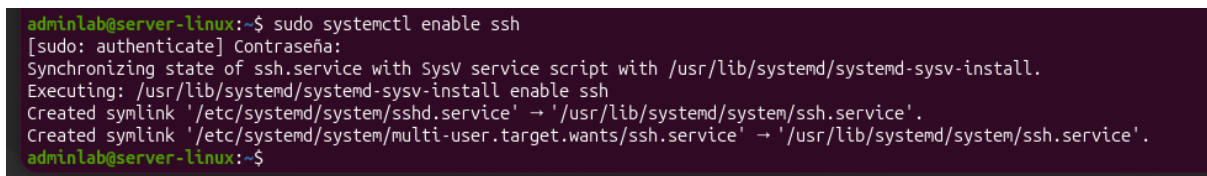
Estas verificaciones permitieron confirmar que el servidor SSH había quedado correctamente instalado y preparado para aceptar conexiones remotas.

Figura 11. Estado del servicio SSH administrado por systemd.



```
Procesando disparadores para ufw (0.30.2-9ubuntu1) ...
adminlab@server-linux:~$ systemctl status ssh
○ ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: enabled)
   Active: inactive (dead)
   TriggeredBy: ● ssh.socket
   Docs: man:sshd(8)
        man:sshd_config(5)
adminlab@server-linux:~$
```

Figura 12. Habilitación del servicio durante el arranque del sistema.



```
adminlab@server-linux:~$ sudo systemctl enable ssh
[sudo: authenticate] Contraseña:
Synchronizing state of ssh.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable ssh
Created symlink '/etc/systemd/system/ssh.service' → '/usr/lib/systemd/system/ssh.service'.
Created symlink '/etc/systemd/system/multi-user.target.wants/ssh.service' → '/usr/lib/systemd/system/ssh.service'.
adminlab@server-linux:~$
```

Figura 13. Puerto TCP 22 en estado LISTEN.

```
adminlab@server-linux:~$ sudo ss -tulnp | grep :22
tcp    LISTEN 0      4096      0.0.0.0:22      0.0.0.0:*    users:((("systemd",pid=1,fd=99))
tcp    LISTEN 0      4096      [::]:22        [::]:*       users:((("systemd",pid=1,fd=100))
adminlab@server-linux:~$
```

Figura 14. Versión instalada de OpenSSH.

```
adminlab@server-linux:~$ ssh -V
OpenSSH_10.2p1 Ubuntu-2ubuntu3.2, OpenSSL 3.5.5 27 Jan 2026
```

Configuración y Endurecimiento del Servicio OpenSSH

Una vez verificado el correcto funcionamiento del servicio OpenSSH, se procedió a realizar una configuración inicial orientada a fortalecer la seguridad del servidor.

El objetivo de esta etapa consistió en reducir la superficie de ataque del servicio SSH antes de iniciar las pruebas experimentales, aplicando medidas preventivas comúnmente utilizadas en la administración de servidores Linux.

La configuración del servicio se realizó modificando el archivo principal:

/etc/ssh/sshd_config

Este archivo contiene los parámetros que determinan el comportamiento del demonio **sshd**, incluyendo las políticas de autenticación, los usuarios autorizados, los tiempos de espera y las restricciones de acceso remoto.

Durante esta etapa se implementarán las siguientes medidas de seguridad:

- Deshabilitación del acceso remoto directo del usuario **root**.
- Restricción del acceso únicamente a usuarios autorizados.
- Configuración del tiempo máximo permitido para la autenticación.
- Limitación del número de intentos fallidos de inicio de sesión.
- Verificación del archivo de configuración antes de reiniciar el servicio.
- Reinicio controlado del servicio para aplicar los cambios.

Estas modificaciones permiten disminuir considerablemente la probabilidad de éxito de ataques automatizados de fuerza bruta, incrementando el nivel de protección del servidor antes de incorporar mecanismos adicionales de detección y respuesta automática.

Restricción de Usuarios Autorizados



Con el objetivo de reducir la exposición del servicio SSH, se restringió el acceso remoto exclusivamente al usuario administrador del laboratorio.

Para ello se incorporó la siguiente directiva dentro del archivo de configuración:

AllowUsers adminlab

De esta manera, el demonio **sshd** rechazará automáticamente cualquier intento de autenticación proveniente de cuentas que no se encuentren expresamente autorizadas.

Esta medida disminuye la superficie de ataque y simplifica el monitoreo de los intentos de acceso registrados durante las pruebas experimentales.

Configuración del Tiempo Máximo de Autenticación

Se configuró un límite temporal para completar el proceso de autenticación mediante el parámetro:

LoginGraceTime 30

Con esta configuración, aquellas conexiones que permanezcan inactivas durante más de treinta segundos serán finalizadas automáticamente por el servidor.

Esta medida evita que múltiples sesiones permanezcan abiertas indefinidamente consumiendo recursos del sistema.

Limitación de Intentos de Autenticación

Como mecanismo adicional de protección se limitó la cantidad máxima de intentos fallidos permitidos durante una misma conexión SSH.

Para ello se configuró el siguiente parámetro:

MaxAuthTries 3

De esta manera, una sesión será terminada automáticamente luego de tres intentos consecutivos de autenticación fallida.

Si bien esta configuración no impide completamente un ataque distribuido de fuerza bruta, reduce significativamente la cantidad de intentos que pueden realizarse sobre una misma conexión, complementando posteriormente el funcionamiento de Fail2Ban.

Implementación del Sistema de Detección Automática mediante Fail2Ban

Una vez finalizada la configuración segura del servicio OpenSSH, se incorporó el componente encargado de realizar la detección automática de intentos reiterados de autenticación fallida.

Para ello se seleccionó **Fail2Ban**, una herramienta ampliamente utilizada en servidores Linux para proteger servicios expuestos a la red frente a ataques automatizados de fuerza bruta.

A diferencia de un firewall tradicional, Fail2Ban no inspecciona directamente el tráfico de red. Su funcionamiento se basa en el análisis continuo de los archivos de registro generados por los distintos servicios del sistema operativo.

En el presente proyecto, Fail2Ban supervisará permanentemente el archivo **/var/log/auth.log**, donde OpenSSH registra cada intento de autenticación.

Cuando una dirección IP supere la cantidad máxima de intentos fallidos permitidos durante un determinado intervalo de tiempo, la herramienta interpretará dicho comportamiento como un posible ataque de fuerza bruta y ejecutará automáticamente una acción de mitigación consistente en el bloqueo temporal de la dirección IP atacante mediante el firewall del sistema.

De esta manera, el servidor adquiere la capacidad de responder automáticamente ante incidentes de seguridad sin requerir intervención del administrador.

Figura 15. Instalación de la herramienta Fail2Ban.

```
adminlab@server-linux:~$ sudo apt update
[sudo: authenticate] Contraseña:
Obj:1 http://ar.archive.ubuntu.com/ubuntu resolute InRelease
Obj:2 http://ar.archive.ubuntu.com/ubuntu resolute-updates InRelease
Obj:3 http://ar.archive.ubuntu.com/ubuntu resolute-backports InRelease
Obj:4 http://security.ubuntu.com/ubuntu resolute-security InRelease
Todos los paquetes están actualizados.
adminlab@server-linux:~$ sudo apt install fail2ban -y
Instalando:
  fail2ban

Instalando dependencias:
  python3-pyasyncore python3-pyinotify whois

Paquetes sugeridos:
  mailx monit sqlite3 python-pyinotify-doc

Resumen:
  Actualizando: 0, Instalando 4, Eliminando: 0, no actualizando: 0
  Tamaño de la descarga: 510 kB
  Espacio necesario: 2.620 kB / 12,1 GB disponible
```

Figura 16. Verificamos el estado de la herramienta.

```
adminlab@server-linux:~$ sudo systemctl status fail2ban
● fail2ban.service - Fail2Ban Service
   Loaded: loaded (/usr/lib/systemd/system/fail2ban.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-06-28 22:39:16 -03; 2min 33s ago
     Invocation: e7c772b2888a4bb7ac84940707445118
       Docs: man:fail2ban(1)
    Main PID: 6978 (fail2ban-server)
      Tasks: 5 (limit: 1582)
     Memory: 17.6M (peak: 18.1M)
        CPU: 313ms
      CGroup: /system.slice/fail2ban.service
              └─6978 /usr/bin/python3 /usr/bin/fail2ban-server -xf start

jun 28 22:39:16 server-linux systemd[1]: Started fail2ban.service - Fail2Ban Service.
jun 28 22:39:17 server-linux fail2ban-server[6978]: Server ready
```

Configuración del Servicio Fail2Ban

Una vez instalado el servicio, se procedió a configurar una política de protección específica para el servicio SSH.

Con el objetivo de preservar la configuración original de la aplicación y facilitar futuras actualizaciones del sistema, no se modificó directamente el archivo `jail.conf`. En su lugar, se creó un archivo de configuración local denominado `jail.local`, siguiendo las recomendaciones de la documentación oficial de Fail2Ban.

En dicho archivo se definieron los parámetros que determinarán el comportamiento del sistema de detección y respuesta automática frente a intentos reiterados de autenticación fallida.

Para el desarrollo del laboratorio se estableció la siguiente política de seguridad:

Parámetro	Valor
Servicio protegido	SSH
Puerto	22
Intentos máximos	3
Ventana de tiempo	2 minutos
Tiempo de bloqueo	1 hora
Backend	systemd

Esta configuración permitirá detectar rápidamente los intentos de fuerza bruta generados durante las pruebas experimentales y aplicar automáticamente un bloqueo temporal sobre la dirección IP atacante.

Figura 17. Configuramos el archivo copiado de configuración **jail.local** en el cual incluimos los parámetros de configuración seleccionados.

```

adminlab@server-linux:~$ sudo nano /etc/fail2ban/jail.local

[sshd]
# To use more aggressive sshd modes set filter parameter "mode" in jail.local:
# normal (default), ddos, extra or aggressive (combines all).
# See "tests/files/logs/sshd" or "filter.d/sshd.conf" for usage example and details.
#mode = normal
enabled = true
port = ssh
logpath = %(sshd_log)s
backend = systemd
maxretry = 3
findtime = 2m

```

Con esta configuración, cualquier dirección IP que falle **tres veces** en un período de **dos minutos** será bloqueada durante **1 hora** (configuración Default de jail.conf).

Figura 18. Chequeamos el estado de la ejecución de fail2ban para el lado cliente, y notamos que - como de esperarse -, aún no se han tomado ningunas de las acciones basadas en nuestros nuevos parámetros.

```

adminlab@server-linux:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
|   |- Currently failed: 0
|   |- Total failed: 0
|   `-- Journal matches: _SYSTEMD_UNIT=ssh.service + _COMM=sshd
`- Actions
    |- Currently banned: 0
    |- Total banned: 0
    `-- Banned IP list:
adminlab@server-linux:~$

```

Implementación del Servicio de Auditoría (auditd)

Con el objetivo de complementar los mecanismos de registro del sistema, se incorporó el servicio **auditd**, perteneciente al subsistema **Linux Audit Framework**.

A diferencia de los registros generados por OpenSSH o por el sistema de logging tradicional, auditd permite registrar eventos de seguridad de manera más detallada, proporcionando información útil para tareas de auditoría e investigación posterior.

Dentro del alcance del presente trabajo, este servicio será utilizado como mecanismo complementario de registro, permitiendo disponer de una fuente adicional de evidencias durante la ejecución de los ataques controlados.

Si bien la detección automática de los intentos de fuerza bruta será realizada por **Fail2Ban**, la incorporación de **auditd** enriquece el laboratorio al demostrar la utilización conjunta de distintos mecanismos nativos del sistema operativo Linux para el monitoreo de eventos de seguridad.

La instalación se realizó mediante el gestor de paquetes APT:

```
sudo apt install auditd audispd-plugins -y
```

Figura 19. Instalación de la herramienta audit y comprobación del estado de su instalación

```
adminlab@server-linux:~$ sudo apt install auditd audispd-plugins -y
Instalando:
  audispd-plugins  auditd

Instalando dependencias:
  libauparse0t64  libauplugi1

Resumen:
  Actualizando: 0, Instalando 4, Eliminando: 0, no actualizando: 7
  Tamaño de la descarga: 338 kB
  Espacio necesario: 1.106 kB / 12,1 GB disponible

Des:1 http://ar.archive.ubuntu.com/ubuntu resolute/main amd64 libauparse0t64 amd64 1:4.1.2-1build1 [63,4 kB]
Des:2 http://ar.archive.ubuntu.com/ubuntu resolute/main amd64 libauplugi1 amd64 1:4.1.2-1build1 [9.978 B]
Des:3 http://ar.archive.ubuntu.com/ubuntu resolute/main amd64 auditd amd64 1:4.1.2-1build1 [224 kB]
Des:4 http://ar.archive.ubuntu.com/ubuntu resolute/universe amd64 audispd-plugins amd64 1:4.1.2-1build1 [40,1 kB]
Descargados 338 kB en 1s (251 kB/s)
Seleccionando el paquete libauparse0t64:amd64 previamente no seleccionado.
(Leyendo la base de datos ... 201410 ficheros o directorios instalados actualmente.)
Preparando para desempaquetar .../libauparse0t64_1%3a4.1.2-1build1_amd64.deb ...
Añadiendo `desviación de /lib/x86_64-linux-gnu/libauparse.so.0 a /lib/x86_64-linux-gnu/libauparse.so.0.usr-is-merged po
```


Finalizada la instalación, se verificó el correcto funcionamiento del servicio mediante las herramientas de administración proporcionadas por systemd.

```
adminlab@server-linux:~$ sudo systemctl status auditd
● auditd.service - Security Audit Logging Service
   Loaded: loaded (/usr/lib/systemd/system/auditd.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-06-29 18:35:23 -03; 1min 26s ago
     Invocation: 7269e62738724833b8605b4a78747202
       Docs: man:auditd(8)
            https://github.com/linux-audit/audit-documentation
   Process: 5065 ExecStart=/usr/sbin/auditd (code=exited, status=0/SUCCESS)
    Main PID: 5067 (auditd)
      Tasks: 2 (limit: 1582)
     Memory: 496K (peak: 2M)
        CPU: 7ms
       CGroup: /system.slice/auditd.service
              └─5067 /usr/sbin/auditd

jun 29 18:35:23 server-linux systemd[1]: Starting auditd.service - Security Audit Logging Service...
jun 29 18:35:23 server-linux auditd[5067]: No plugins found, not dispatching events
jun 29 18:35:23 server-linux auditd[5067]: Init complete, auditd 4.1.2 listening for events (startup state enable)
jun 29 18:35:23 server-linux systemd[1]: Started auditd.service - Security Audit Logging Service.
adminlab@server-linux:~$
```

Figura 20. Se verifica la versión instalada del programa y el estado del subsistema de auditoría

```
adminlab@server-linux:~$ sudo auditctl -v
auditctl version 4.1.2
adminlab@server-linux:~$
```

```
adminlab@server-linux:~$ sudo auditctl -s
enabled 1
failure 1
pid 5067
rate_limit 0
backlog_limit 8192
lost 0
backlog 0
backlog_wait_time 60000
backlog_wait_time_actual 0
loginuid_immutable 0 unlocked
adminlab@server-linux:~$
```

Fase 5 - Planeación del ataque

Construcción de la Máquina Atacante

Con el objetivo de reproducir un escenario realista de pruebas de seguridad, se incorporó una segunda máquina virtual destinada a representar al atacante dentro del laboratorio.

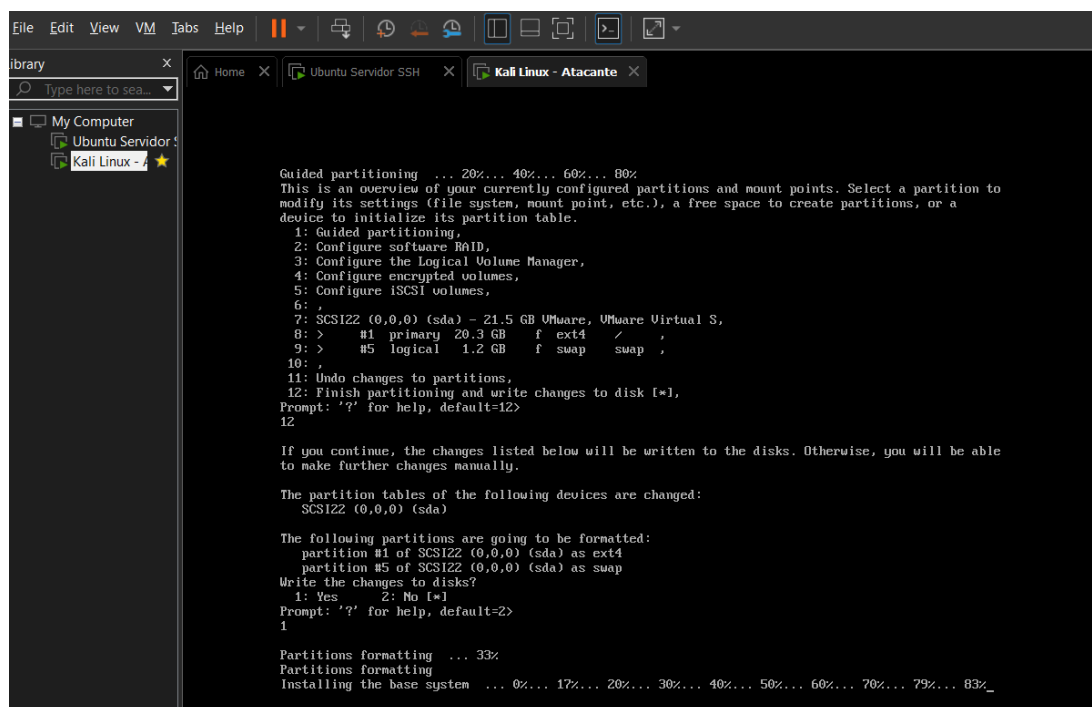
Para ello se seleccionó **Kali Linux**, una distribución basada en Debian ampliamente utilizada en auditorías de seguridad informática, análisis forense y pruebas de penetración. Su elección responde a que incorpora de forma nativa una gran cantidad de herramientas especializadas para la evaluación de infraestructuras y servicios de red.

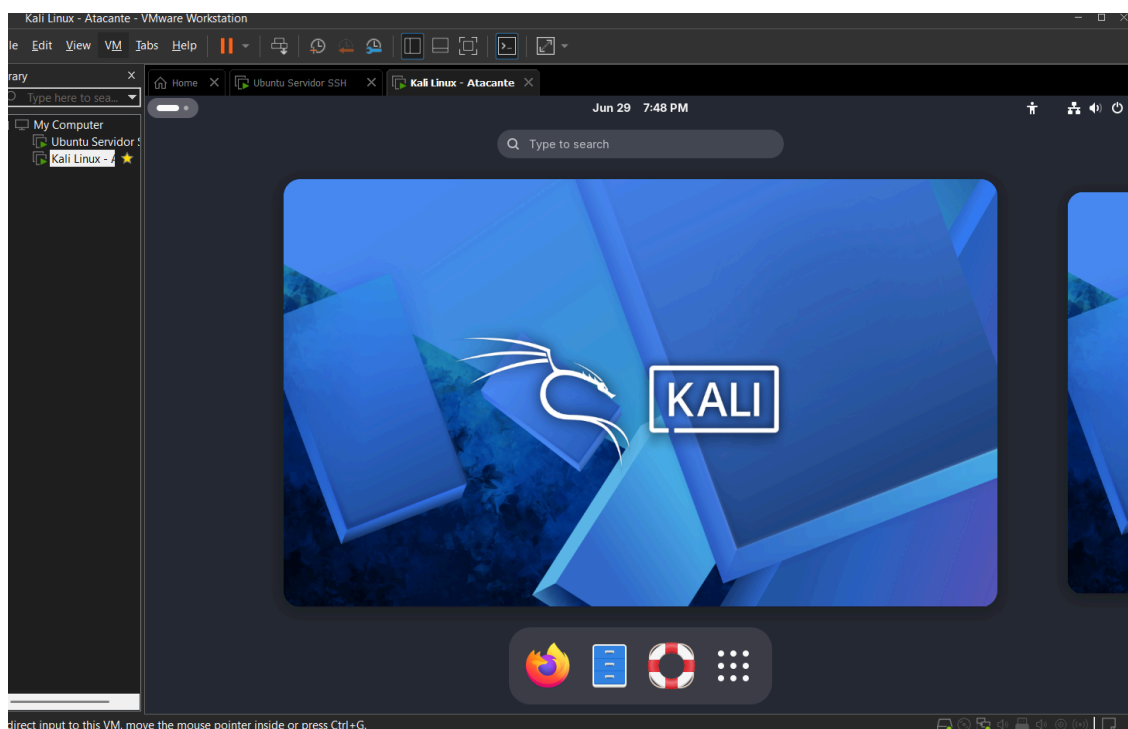
Dentro del alcance del presente Trabajo Práctico Integrador, la máquina Kali Linux será utilizada exclusivamente para generar ataques controlados sobre el servidor Ubuntu previamente configurado, permitiendo validar el funcionamiento del sistema automatizado de detección y respuesta implementado.

Durante esta etapa no se persigue comprometer la integridad del servidor, sino generar un volumen controlado de intentos de autenticación fallida que permitan observar el comportamiento de los distintos mecanismos de protección incorporados al sistema.

Para la creación de la máquina virtual se utilizó VMware Workstation, asignando recursos compatibles con las necesidades del laboratorio.

Figura 31. Creación de la máquina virtual Kali Linux en VMware Workstation.





Preparación del Entorno de Ataque

Una vez finalizada la implementación del servidor Linux protegido, se procedió a preparar el entorno desde el cual se ejecutarían las pruebas experimentales de seguridad.

Para ello se instaló una segunda máquina virtual con **Kali Linux**, destinada exclusivamente a representar al atacante dentro del laboratorio. La utilización de una máquina independiente permitió reproducir un escenario más cercano a un entorno real, diferenciando claramente los roles de atacante y servidor objetivo.

Al igual que en el servidor Ubuntu, se realizó una actualización completa del sistema operativo antes de comenzar la instalación y verificación de las herramientas necesarias para el desarrollo del laboratorio.

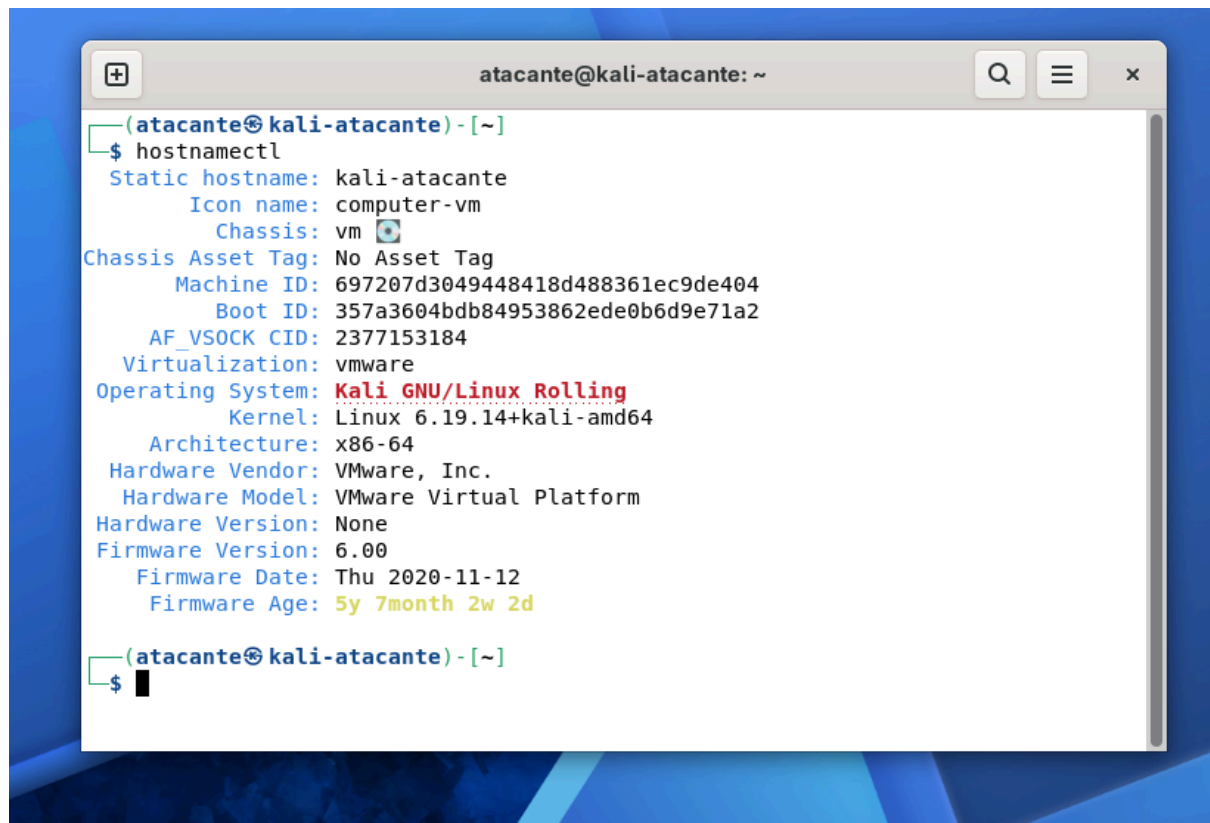
Durante esta etapa también se relevó la información general del sistema operativo, incluyendo la versión de la distribución, la versión del kernel, la arquitectura de hardware y la configuración inicial de la interfaz de red, con el propósito de documentar completamente el entorno experimental.

Las principales características de la máquina atacante se resumen a continuación:

Característica	Valor
Sistema Operativo	Kali Linux
Rol	Máquina atacante
Hipervisor	VMware Workstation
Memoria RAM	4 GB
Procesadores	2 vCPU
Red	NAT (fase de preparación)

Esta configuración proporciona un entorno suficiente para ejecutar herramientas de reconocimiento y pruebas de autenticación sin introducir una carga significativa sobre el equipo anfitrión.

Figura 32. Información general del sistema Kali Linux.



```

(atacante@kali-atacante) - [~]
$ hostnamectl
  Static hostname: kali-atacante
    Icon name: computer-vm
    Chassis: vm
Chassis Asset Tag: No Asset Tag
  Machine ID: 697207d3049448418d488361ec9de404
    Boot ID: 357a3604bdb84953862ede0b6d9e71a2
  AF_VSOCK CID: 2377153184
  Virtualization: vmware
Operating System: Kali GNU/Linux Rolling
    Kernel: Linux 6.19.14+kali-amd64
    Architecture: x86_64
    Hardware Vendor: VMware, Inc.
    Hardware Model: VMware Virtual Platform
    Hardware Version: None
    Firmware Version: 6.00
    Firmware Date: Thu 2020-11-12
    Firmware Age: 5y 7month 2w 2d
(atacante@kali-atacante) - [~]
$
  
```

```
(atacante@kali-atacante)-[~]
$ lsb_release -a
No LSB modules are available.
Distributor ID: Kali
Description:    Kali GNU/Linux Rolling
Release:       2026.2
Codename:      kali-rolling

(atacante@kali-atacante)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:b0:7a:a0 brd ff:ff:ff:ff:ff:ff
    inet 192.168.247.129/24 brd 192.168.247.255 scope global dynamic noprefixroute eth0
        valid_lft 1492sec preferred_lft 1492sec
    inet6 fe80::20c:29ff:feb0:7aa0/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

(atacante@kali-atacante)-[~]
$
```

Warning: Problem unlinking the file /var/cache/apt/srcpkgcache.bin - RemoveCaches (13: Permission denied)

```
(atacante@kali-atacante)-[~]
$ sudo apt update
[sudo] password for atacante:
Notice: It seems that you don't have any APT data sources configured.
Notice: You won't be able to update your system or install new packages.
Notice: For more information, please refer to the online documentation at:
Notice: https://www.kali.org/docs/general-use/kali-apt-sources/
All packages are up to date.

(atacante@kali-atacante)-[~]
$ sudo apt upgrade
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0

(atacante@kali-atacante)-[~]
$
```

Durante la preparación del laboratorio se decidió mantener ambas máquinas virtuales conectadas inicialmente mediante una red de tipo NAT. Esta decisión permitió acceder temporalmente a Internet para instalar paquetes, descargar actualizaciones y verificar el correcto funcionamiento de las herramientas utilizadas durante el proyecto.

Una vez finalizada la preparación de ambos sistemas, la configuración de red será modificada para utilizar una red privada exclusiva del laboratorio, garantizando que las pruebas de fuerza bruta se desarrollen en un entorno completamente aislado y sin impacto sobre otras redes.

Verificación de las Herramientas de Seguridad

Con el objetivo de garantizar la disponibilidad de las herramientas necesarias para el desarrollo de los experimentos, se verificó la instalación de las principales aplicaciones de auditoría incluidas en Kali Linux.

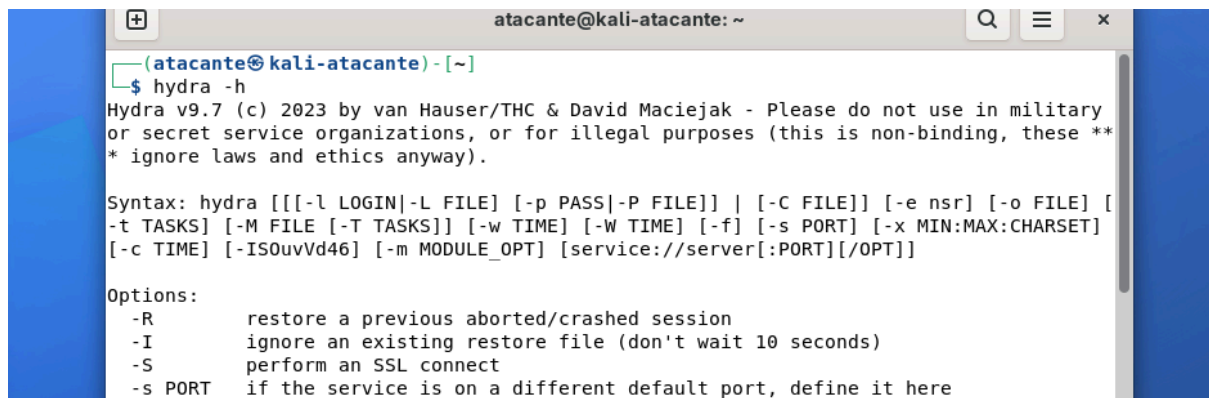
Las herramientas seleccionadas fueron:

- **Hydra**, utilizada para realizar pruebas controladas de autenticación sobre el servicio SSH.
- **Nmap**, utilizada para verificar la disponibilidad del servicio SSH y efectuar tareas básicas de reconocimiento del servidor.

En caso de ser necesario, ambas aplicaciones fueron instaladas o actualizadas mediante el gestor de paquetes APT.

La verificación de estas herramientas permitió asegurar que el entorno atacante se encontraba completamente preparado para iniciar las pruebas experimentales descritas en el capítulo siguiente.

Figura 33. Verificación de la versión instalada de Hydra.



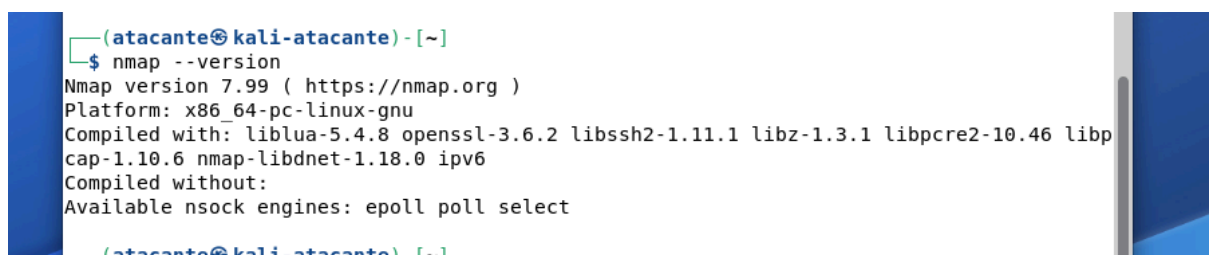
```

atacante@kali-atacante: ~
(atacante@kali-atacante) - [~]
$ hydra -h
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military
or secret service organizations, or for illegal purposes (this is non-binding, these **
* ignore laws and ethics anyway).

Syntax: hydra [[[-l LOGIN|-L FILE] [-p PASS|-P FILE]] | [-C FILE]] [-e nsr] [-o FILE] [
-t TASKS] [-M FILE [-T TASKS]] [-w TIME] [-W TIME] [-f] [-s PORT] [-x MIN:MAX:CHARSET]
[-c TIME] [-ISOuvVd46] [-m MODULE_OPT] [service://server[:PORT][[/OPT]]]

Options:
-R      restore a previous aborted/crashed session
-I      ignore an existing restore file (don't wait 10 seconds)
-S      perform an SSL connect
-s PORT if the service is on a different default port, define it here
  
```

Figura 33. Verificación de la versión instalada de Nmap.



```

(atacante@kali-atacante) - [~]
$ nmap --version
Nmap version 7.99 ( https://nmap.org )
Platform: x86_64-pc-linux-gnu
Compiled with: liblua-5.4.8 openssl-3.6.2 libssh2-1.11.1 libz-1.3.1 libpcrc2-10.46 libp
cap-1.10.6 nmap-libdnet-1.18.0 ipv6
Compiled without:
Available nsock engines: epoll poll select
(atacante@kali-atacante) - [~]
  
```

Fase 6 - Validación Experimental

Una vez finalizada la implementación de la infraestructura del laboratorio, se procedió a ejecutar una serie de pruebas experimentales destinadas a validar el funcionamiento del sistema automatizado de detección y respuesta ante intentos de fuerza bruta.



Los experimentos fueron diseñados con el objetivo de verificar el comportamiento conjunto de los distintos componentes implementados durante el proyecto, incluyendo el servicio OpenSSH, el sistema de registros del sistema operativo Linux, el mecanismo de protección proporcionado por Fail2Ban y las herramientas de auditoría incorporadas al servidor.

Cada escenario experimental fue desarrollado de manera progresiva, comenzando por la validación de la conectividad entre las máquinas virtuales y finalizando con la ejecución de un ataque controlado de fuerza bruta sobre el servicio SSH.

Durante cada prueba se registraron las evidencias obtenidas mediante capturas de pantalla y registros del sistema, permitiendo posteriormente analizar el comportamiento de la arquitectura implementada y verificar el cumplimiento de los objetivos establecidos para el presente Trabajo Práctico Integrador.

Escenario Experimental N.º 1 – Validación de la Conectividad del Laboratorio

Objetivo

Verificar el correcto funcionamiento de la infraestructura de red del laboratorio antes de iniciar las pruebas de seguridad.

Metodología

Finalizada la etapa de implementación, ambas máquinas virtuales fueron desconectadas de la red NAT utilizada durante la instalación de software y configuradas para operar dentro de una red privada exclusiva del laboratorio.

Esta modificación tuvo como finalidad garantizar que los experimentos se desarrollaran en un entorno completamente aislado, evitando que el tráfico generado por las herramientas de auditoría pudiera alcanzar redes externas.

Posteriormente se verificó la comunicación entre ambas máquinas utilizando herramientas básicas de diagnóstico de red.

Resultado esperado

Se espera que ambas máquinas puedan comunicarse correctamente mediante el protocolo ICMP y que el servicio SSH del servidor permanezca accesible desde la máquina atacante.

Procedimiento

Se modificó la configuración de red de ambas máquinas virtuales reemplazando la conexión NAT utilizada durante la fase de implementación por una red privada compartida.

Una vez iniciados ambos sistemas, se verificó la asignación de direcciones IP y posteriormente se comprobó la conectividad mediante solicitudes ICMP (ping) en ambos sentidos.

Finalmente, se realizó una prueba preliminar de conexión al servicio SSH desde la máquina Kali Linux, confirmando que el servidor aceptaba conexiones remotas dentro de la red del laboratorio.

Resultados obtenidos

Las pruebas de conectividad confirmaron que ambas máquinas virtuales podían intercambiar tráfico correctamente dentro de la red privada.

Asimismo, el servidor respondió satisfactoriamente a las solicitudes de conexión sobre el puerto TCP 22, indicando que el servicio OpenSSH permanecía operativo luego del cambio de configuración de red.

Figura 35. Configuración de la red privada del laboratorio.

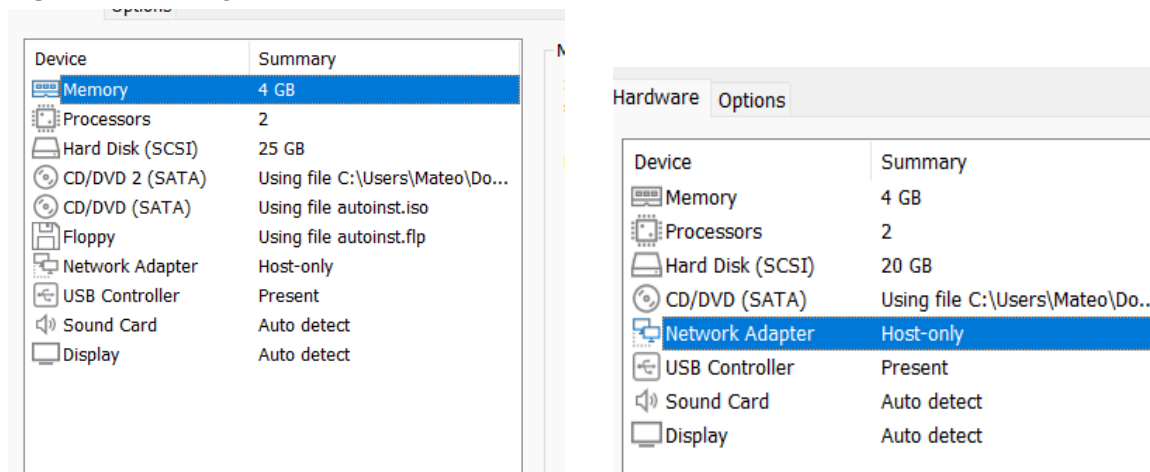


Figura 36. Verificación de la conectividad mediante ICMP.

```
adminlab@server-linux:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:5f:e4:59 brd ff:ff:ff:ff:ff:ff
    inet 192.168.132.128/24 brd 192.168.132.255 scope global dynamic noprefixroute ens33
        valid_lft 1716sec preferred_lft 1716sec
    inet6 fe80::20c:29ff:fe5f:e459/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
adminlab@server-linux:~$
```



```
(atacante@kali-atacante) - [~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:b0:7a:a0 brd ff:ff:ff:ff:ff:ff
    inet 192.168.132.129/24 brd 192.168.132.255 scope global dynamic noprefixroute eth0
        valid_lft 1687sec preferred_lft 1687sec
```

Comprobamos la conectividad entre ambas maquinas..

```
(atacante@kali-atacante) - [~]
$ ping 192.168.132.128
PING 192.168.132.128 (192.168.132.128) 56(84) bytes of data.
64 bytes from 192.168.132.128: icmp_seq=1 ttl=64 time=1.12 ms
64 bytes from 192.168.132.128: icmp_seq=2 ttl=64 time=1.80 ms
64 bytes from 192.168.132.128: icmp_seq=3 ttl=64 time=1.14 ms
64 bytes from 192.168.132.128: icmp_seq=4 ttl=64 time=0.873 ms
64 bytes from 192.168.132.128: icmp_seq=5 ttl=64 time=0.390 ms
64 bytes from 192.168.132.128: icmp_seq=6 ttl=64 time=0.022 ms

adminlab@server-linux:~$ ping 192.168.132.129
PING 192.168.132.129 (192.168.132.129) 56(84) bytes of data.
64 bytes from 192.168.132.129: icmp_seq=1 ttl=64 time=0.576 ms
64 bytes from 192.168.132.129: icmp_seq=2 ttl=64 time=1.56 ms
64 bytes from 192.168.132.129: icmp_seq=3 ttl=64 time=1.85 ms
64 bytes from 192.168.132.129: icmp_seq=4 ttl=64 time=1.43 ms
64 bytes from 192.168.132.129: icmp_seq=5 ttl=64 time=0.821 ms
^C
--- 192.168.132.129 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4008ms
rtt min/avg/max/mdev = 0.576/1.246/1.853/0.474 ms
adminlab@server-linux:~$
```

Figura 37. Primera conexión SSH desde Kali Linux.

```
(atacante@kali-atacante) - [~]
$ ssh adminlab@192.168.132.128
The authenticity of host '192.168.132.128 (192.168.132.128)' can't be established.
ED25519 key fingerprint is: SHA256:+aFieubrdq3H8v4oYnVCgzZsEH4cftozLUvyhr6pfwY
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.132.128' (ED25519) to the list of known hosts.
adminlab@192.168.132.128's password: █
```

Escenario Experimental N.º 2 – Reconocimiento del Servidor mediante Nmap

Objetivo

Verificar que el servidor Linux expone el servicio SSH a la red del laboratorio y obtener información básica sobre dicho servicio desde la perspectiva de un potencial atacante.



Metodología

Como primera etapa de una prueba de penetración resulta habitual realizar tareas de reconocimiento con el propósito de identificar los servicios disponibles sobre el sistema objetivo.

Para ello se utilizó la herramienta **Nmap (Network Mapper)**, ampliamente utilizada para el descubrimiento de hosts, identificación de puertos abiertos y detección de servicios de red.

Durante este experimento se efectuó un escaneo dirigido exclusivamente al servidor Ubuntu, con el fin de comprobar la disponibilidad del servicio SSH previamente configurado.

Procedimiento

Desde la máquina Kali Linux se ejecutó inicialmente un escaneo básico sobre el servidor utilizando el siguiente comando:

```
nmap 192.168.132.128
```

Posteriormente se realizó un escaneo específico sobre el puerto correspondiente al servicio SSH:

```
nmap -p 22 192.168.132.128
```

Finalmente se ejecutó un análisis más detallado con detección de versiones de servicios:

```
nmap -sV -p 22 192.168.132.128
```

Esta última prueba permitió identificar la versión del servidor OpenSSH ejecutándose sobre Ubuntu.

Resultados Esperados

Se espera obtener los siguientes resultados:

- El servidor responde a las solicitudes de escaneo.
- El puerto TCP 22 aparece en estado **open**.
- Nmap identifica correctamente el servicio OpenSSH.
- Se obtiene información suficiente para planificar el escenario de ataque posterior.

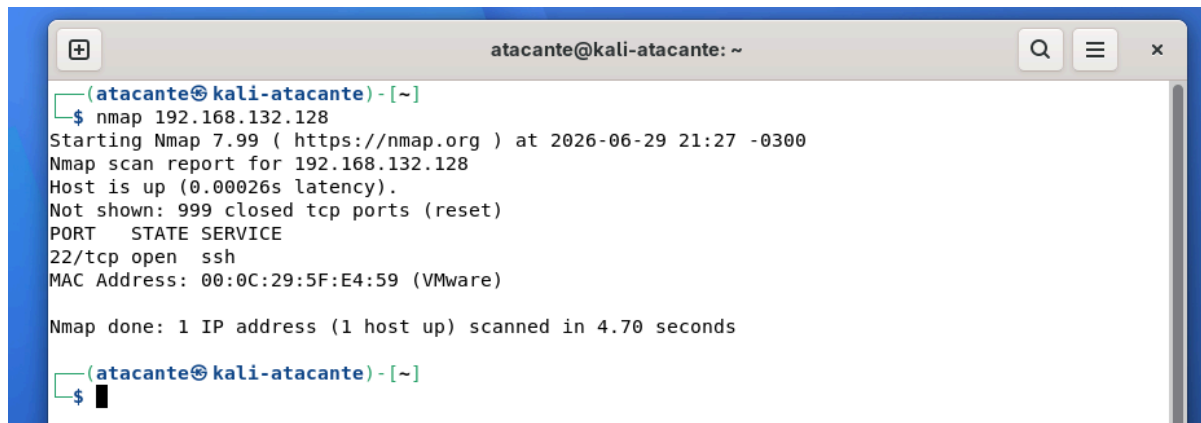
Resultados Obtenidos

Los resultados obtenidos confirmaron que el servidor exponía correctamente el servicio SSH sobre el puerto TCP 22.

Asimismo, Nmap logró identificar el servicio OpenSSH y obtener información relativa a la versión instalada, demostrando que un potencial atacante puede reconocer el servicio disponible antes de iniciar un intento de autenticación.

Estos resultados validan que el entorno del laboratorio reproduce adecuadamente un escenario realista de exposición de un servicio de administración remota.

Figura 38. Escaneo inicial del servidor mediante Nmap.

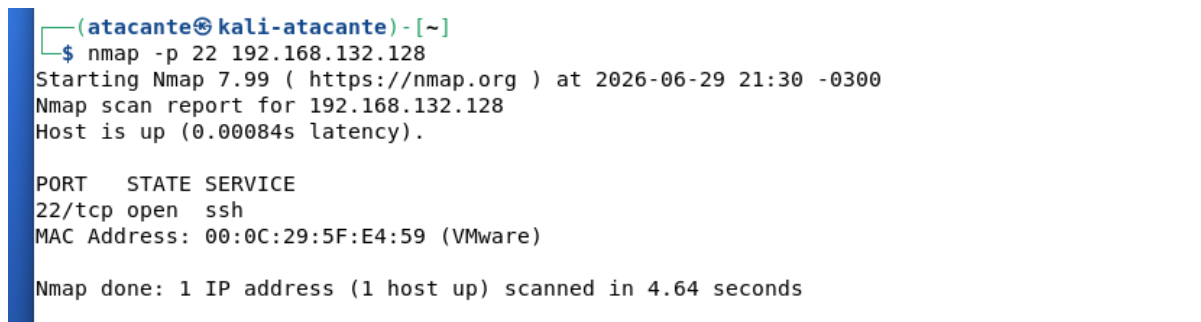


```
(atacante@kali-atacante) - [~]
$ nmap 192.168.132.128
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-29 21:27 -0300
Nmap scan report for 192.168.132.128
Host is up (0.00026s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
MAC Address: 00:0C:29:5F:E4:59 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 4.70 seconds

(atacante@kali-atacante) - [~]
$
```

Figura 39. Verificación del puerto TCP 22.

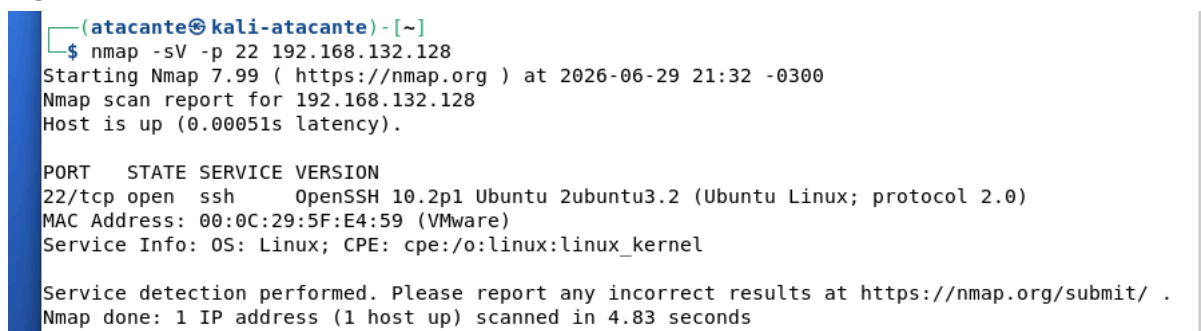


```
(atacante@kali-atacante) - [~]
$ nmap -p 22 192.168.132.128
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-29 21:30 -0300
Nmap scan report for 192.168.132.128
Host is up (0.00084s latency).

PORT      STATE SERVICE
22/tcp    open  ssh
MAC Address: 00:0C:29:5F:E4:59 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 4.64 seconds
```

Figura 40. Detección del servicio OpenSSH mediante `nmap -sV`.



```
(atacante@kali-atacante) - [~]
$ nmap -sV -p 22 192.168.132.128
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-29 21:32 -0300
Nmap scan report for 192.168.132.128
Host is up (0.00051s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.2p1 Ubuntu 2ubuntu3.2 (Ubuntu Linux; protocol 2.0)
MAC Address: 00:0C:29:5F:E4:59 (VMware)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 4.83 seconds
```

Escenario Experimental N.º 3 – Simulación de un Ataque de Fuerza Bruta

Objetivo



Evaluar el funcionamiento del sistema automatizado de detección y respuesta implementado sobre el servidor Linux mediante la simulación de un ataque de fuerza bruta dirigido contra el servicio OpenSSH, verificando la capacidad de Fail2Ban para detectar los intentos reiterados de autenticación fallida y bloquear automáticamente la dirección IP del atacante.

Metodología

Con el entorno del laboratorio previamente configurado y el servicio OpenSSH en funcionamiento, se procedió a ejecutar un ataque controlado desde la máquina Kali Linux utilizando la herramienta Hydra.

El ataque fue dirigido hacia la cuenta administrativa **adminlab**, creada durante las etapas iniciales del laboratorio, empleando un diccionario de contraseñas de prueba compuesto exclusivamente por credenciales incorrectas.

Durante la ejecución del experimento se monitoreó simultáneamente el comportamiento de Hydra, los registros generados por OpenSSH y el estado del servicio Fail2Ban, con el propósito de verificar la interacción entre todos los componentes de seguridad implementados.

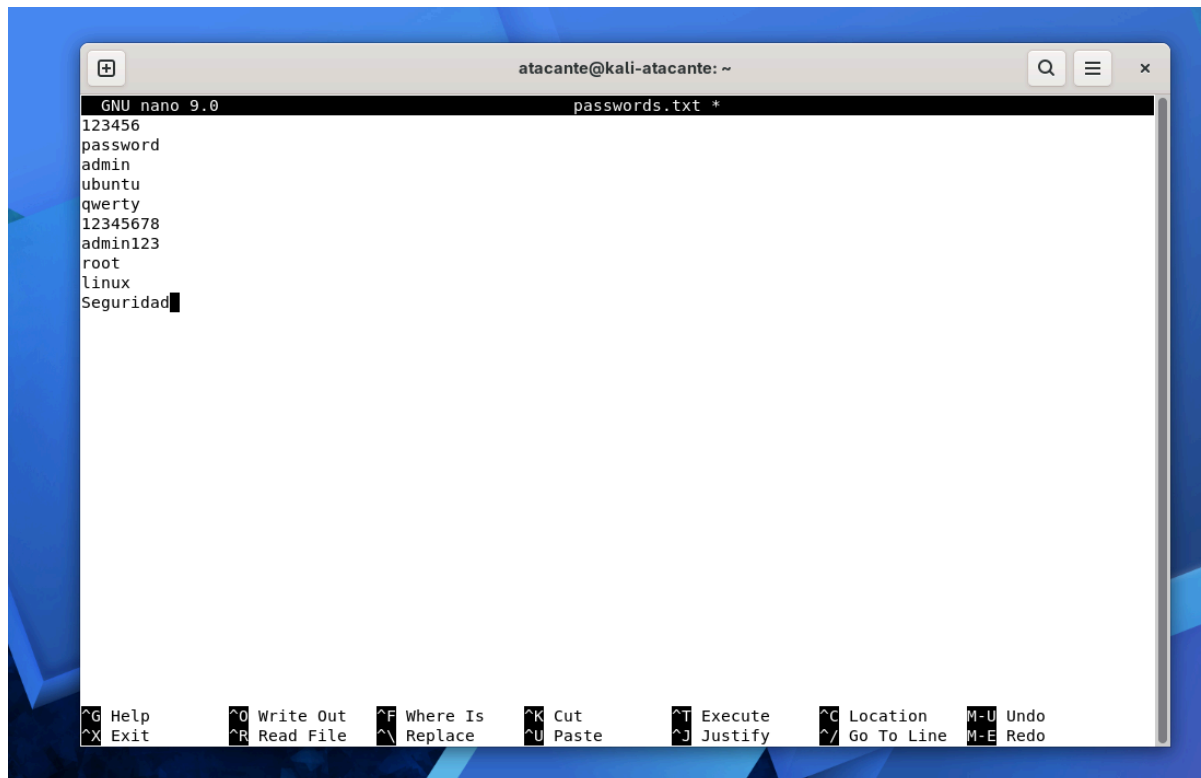
Preparación del escenario

Preparación del diccionario de contraseñas

Como paso previo al ataque, se creó un archivo denominado **passwords.txt** que contenía un conjunto de contraseñas incorrectas utilizadas para simular un ataque automatizado de fuerza bruta.

El archivo fue creado mediante el editor Nano en la máquina Kali Linux y posteriormente utilizado como entrada para Hydra.

Figura 41. Creación del archivo `passwords.txt`.



Verificación de Hydra

Antes de iniciar el ataque se comprobó la disponibilidad de la herramienta Hydra y su correcto funcionamiento mediante la consulta de su versión instalada.

Esta verificación permitió asegurar que el entorno del atacante se encontraba correctamente preparado para el desarrollo del experimento.

Figura 47. Verificación de la instalación de Hydra.

Ejecución del ataque

El ataque fue ejecutado mediante el siguiente comando:

```
hydra -V -t 1 -l adminlab -P passwords.txt ssh://192.168.132.128
```

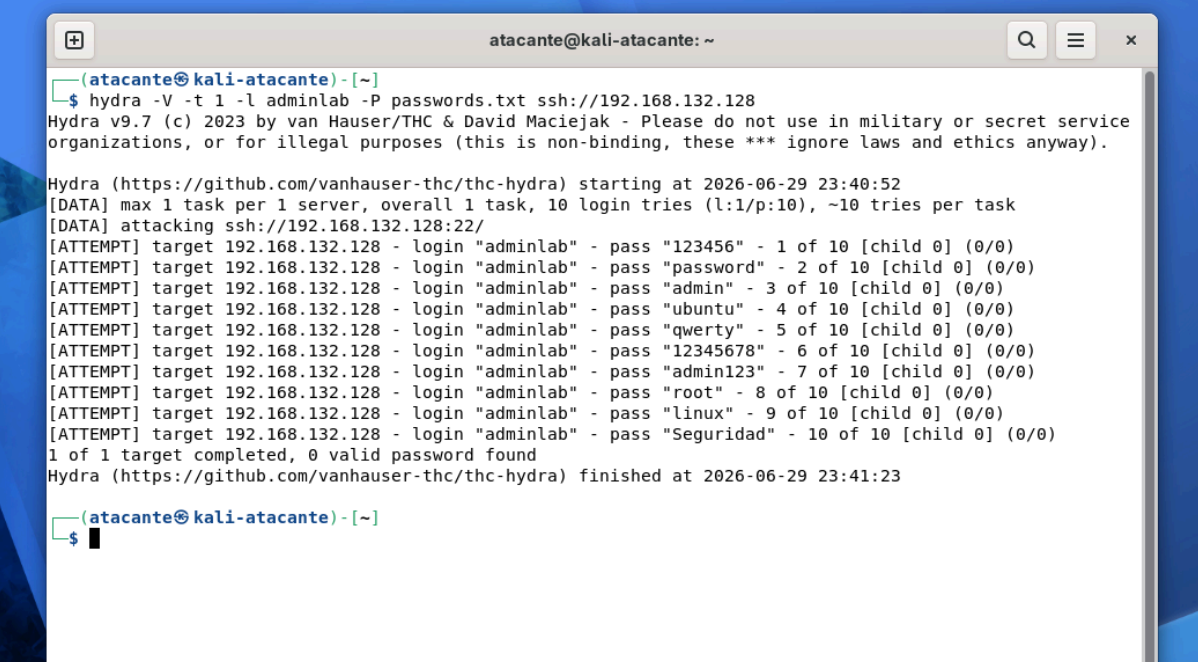
Donde:

- `-V` muestra cada intento de autenticación realizado.

- `-t 1` limita la ejecución a una única conexión simultánea, facilitando el seguimiento de los registros generados por el servidor.
- `-l adminlab` especifica el usuario objetivo.
- `-P passwords.txt` indica el diccionario de contraseñas utilizado.

Durante la ejecución se observaron sucesivos intentos de autenticación sobre el servicio SSH, los cuales fueron rechazados por el servidor al no corresponderse con la contraseña válida del usuario.

Figura 42. Ejecución del ataque de fuerza bruta mediante Hydra.



```

(atacante@kali-atacante)-[~]
$ hydra -V -t 1 -l adminlab -P passwords.txt ssh://192.168.132.128
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service
organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-06-29 23:40:52
[DATA] max 1 task per 1 server, overall 1 task, 10 login tries (l:1/p:10), ~10 tries per task
[DATA] attacking ssh://192.168.132.128:22/
[ATTEMPT] target 192.168.132.128 - login "adminlab" - pass "123456" - 1 of 10 [child 0] (0/0)
[ATTEMPT] target 192.168.132.128 - login "adminlab" - pass "password" - 2 of 10 [child 0] (0/0)
[ATTEMPT] target 192.168.132.128 - login "adminlab" - pass "admin" - 3 of 10 [child 0] (0/0)
[ATTEMPT] target 192.168.132.128 - login "adminlab" - pass "ubuntu" - 4 of 10 [child 0] (0/0)
[ATTEMPT] target 192.168.132.128 - login "adminlab" - pass "qwerty" - 5 of 10 [child 0] (0/0)
[ATTEMPT] target 192.168.132.128 - login "adminlab" - pass "12345678" - 6 of 10 [child 0] (0/0)
[ATTEMPT] target 192.168.132.128 - login "adminlab" - pass "admin123" - 7 of 10 [child 0] (0/0)
[ATTEMPT] target 192.168.132.128 - login "adminlab" - pass "root" - 8 of 10 [child 0] (0/0)
[ATTEMPT] target 192.168.132.128 - login "adminlab" - pass "linux" - 9 of 10 [child 0] (0/0)
[ATTEMPT] target 192.168.132.128 - login "adminlab" - pass "Seguridad" - 10 of 10 [child 0] (0/0)
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-06-29 23:41:23

(atacante@kali-atacante)-[~]
$
  
```

Verificación del funcionamiento de OpenSSH

Mientras Hydra realizaba los intentos de autenticación, se monitoreó en tiempo real el servicio OpenSSH utilizando `journalctl`.

Los registros evidenciaron múltiples eventos de autenticación fallida correspondientes al usuario **adminlab**, confirmando que el servidor registraba correctamente cada intento de acceso realizado por la herramienta de fuerza bruta.

Asimismo, se observó el mensaje generado por el módulo PAM indicando que el número máximo de intentos permitidos dentro de una misma sesión había sido superado.

Este comportamiento constituye un mecanismo propio del servicio OpenSSH destinado a limitar los intentos de autenticación dentro de una misma conexión.

Figura 43. Registros de autenticación fallida obtenidos mediante `journalctl`.

```
adminlab@server-linux:~$ sudo journalctl -f -u ssh
[sudo: authenticate] Contraseña:
jun 29 23:41:13 server-linux sshd-session[4311]: Failed password for adminlab from 192.168.132.129 port 56476 ssh2
jun 29 23:41:14 server-linux unix_chkpwd[4316]: password check failed for user (adminlab)
jun 29 23:41:16 server-linux sshd-session[4311]: Failed password for adminlab from 192.168.132.129 port 56476 ssh2
jun 29 23:41:17 server-linux unix_chkpwd[4317]: password check failed for user (adminlab)
jun 29 23:41:18 server-linux sshd-session[4311]: Failed password for adminlab from 192.168.132.129 port 56476 ssh2
jun 29 23:41:18 server-linux unix_chkpwd[4319]: password check failed for user (adminlab)
jun 29 23:41:20 server-linux sshd-session[4311]: Failed password for adminlab from 192.168.132.129 port 56476 ssh2
jun 29 23:41:21 server-linux sshd-session[4311]: Connection closed by authenticating user adminlab 192.168.132.129 port 56476 [preauth]
jun 29 23:41:21 server-linux sshd-session[4311]: PAM 4 more authentication failures; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.132.129 user=adminlab
jun 29 23:41:21 server-linux sshd-session[4311]: PAM service(sshd) ignoring max retries; 5 > 3
```

Verificación del funcionamiento de Fail2Ban

Una vez registrados los intentos fallidos, se consultó el estado del *jail* correspondiente al servicio SSH mediante la utilidad `fail2ban-client`.

Inicialmente se observó que Fail2Ban comenzaba a contabilizar los intentos de autenticación detectados sobre el servicio SSH.

Posteriormente, una vez procesados los eventos registrados por OpenSSH, el servicio incorporó automáticamente la dirección IP de la máquina Kali Linux a la lista de direcciones bloqueadas.

La información obtenida confirmó que el mecanismo de detección funcionó correctamente y que la política de seguridad configurada fue aplicada de manera automática.

Figura 44. Estado del *jail* `sshd` antes de la aplicación del bloqueo.

```
adminlab@server-linux:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
| |- Currently failed: 1
| |- Total failed: 2
| \- Journal matches: _SYSTEMD_UNIT=ssh.service + _COMM=sshd
- Actions
  |- Currently banned: 0
  |- Total banned: 0
  \- Banned IP list:
adminlab@server-linux:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
| |- Currently failed: 1
| |- Total failed: 2
| \- Journal matches: _SYSTEMD_UNIT=ssh.service + _COMM=sshd
- Actions
  |- Currently banned: 0
  |- Total banned: 0
  \- Banned IP list:
adminlab@server-linux:~$
```

Figura 45. Dirección IP atacante incorporada a la lista de direcciones bloqueadas.

```
^[[A^Cadminlab@server-linux:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
|   |- Currently failed: 0
|   |- Total failed:     12
|   `-- Journal matches: _SYSTEMD_UNIT=ssh.service + _COMM=sshd
`- Actions
    |- Currently banned: 1
    |- Total banned:     1
    `-- Banned IP list:  192.168.132.129
```

Validación del bloqueo

Como comprobación final del experimento se intentó establecer nuevamente una conexión SSH desde la máquina atacante hacia el servidor Ubuntu.

En esta oportunidad el servidor rechazó inmediatamente la conexión, impidiendo el establecimiento de una nueva sesión remota.

Este comportamiento confirmó que la dirección IP del atacante había sido bloqueada correctamente por Fail2Ban como consecuencia de los múltiples intentos fallidos de autenticación.

Figura 46. Intento de conexión SSH posterior al bloqueo automático.

```
(atacante@kali-atacante) - [~]
$ sudo ssh adminlab@192.168.132.128
[sudo] password for atacante:
ssh: connect to host 192.168.132.128 port 22: Connection refused

(atacante@kali-atacante) - [~]
$ sudo ssh adminlab@192.168.132.128
ssh: connect to host 192.168.132.128 port 22: Connection refused

(atacante@kali-atacante) - [~]
$ sudo ssh adminlab@192.168.132.128
ssh: connect to host 192.168.132.128 port 22: Connection refused

(atacante@kali-atacante) - [~]
$ sudo ssh adminlab@192.168.132.128
ssh: connect to host 192.168.132.128 port 22: Connection refused

(atacante@kali-atacante) - [~]
```


Fase 7 - Análisis de Evidencias y Correlación de Eventos

Finalizada la ejecución de los escenarios experimentales, se procedió al análisis de las evidencias obtenidas con el propósito de reconstruir la secuencia completa de eventos ocurridos durante el ataque de fuerza bruta.

El análisis permitió correlacionar la información registrada por los distintos componentes del sistema operativo Linux, identificando la interacción existente entre el servicio OpenSSH, el subsistema de autenticación PAM, el sistema de registros (`systemd-journal`) y el mecanismo de protección implementado mediante Fail2Ban.

Esta etapa resulta de especial importancia, ya que demuestra el funcionamiento integrado de la arquitectura diseñada para el laboratorio y permite comprender el comportamiento interno del sistema operativo frente a un intento de acceso no autorizado.

Registro de los Intentos de Autenticación

Durante la ejecución del ataque automatizado, el servicio OpenSSH recibió múltiples solicitudes consecutivas de autenticación provenientes de la máquina atacante.

Cada intento fue procesado por el subsistema de autenticación del sistema operativo y registrado automáticamente en los registros del servicio SSH.

El monitoreo realizado mediante `journalctl` permitió verificar que cada intento fallido generó una entrada específica indicando el usuario objetivo, la dirección IP de origen y el resultado de la autenticación.

Entre los eventos registrados se observaron múltiples mensajes del tipo:

Failed password for adminlab from 192.168.132.129

Estas entradas constituyen la principal fuente de información utilizada posteriormente por Fail2Ban para identificar el comportamiento anómalo asociado a un ataque de fuerza bruta.

Ver Figura 43.

Análisis del subsistema PAM

Durante el análisis de los registros del sistema también se observó la intervención del módulo PAM (Pluggable Authentication Modules), responsable de gestionar los procesos de autenticación de usuarios en sistemas Linux.

Uno de los mensajes registrados fue:

```
PAM service(sshd) ignoring max retries; 5 > 3
```

Este evento indica que el servicio SSH detectó que el número de intentos de autenticación dentro de una misma sesión había superado el límite interno definido por el sistema.

Es importante destacar que este mecanismo pertenece al propio proceso de autenticación administrado por PAM y constituye una capa adicional de protección independiente del funcionamiento de Fail2Ban.

La presencia de este mensaje evidencia que el sistema operativo aplicó controles de autenticación incluso antes de que el mecanismo de bloqueo automático analizara los registros generados.

Ver Figura 43.

Análisis de Fail2Ban

Mientras OpenSSH registraba los intentos fallidos de autenticación, el servicio Fail2Ban monitoreó continuamente dichos registros utilizando el backend basado en `systemd`.

Al detectar múltiples eventos compatibles con un ataque de fuerza bruta, el servicio incrementó el contador interno asociado a la dirección IP atacante.

Una vez superado el umbral de intentos definido mediante el parámetro `maxretry`, Fail2Ban ejecutó automáticamente la acción de protección configurada para el servicio SSH.

Como resultado, la dirección IP de la máquina Kali Linux fue incorporada a la lista de direcciones bloqueadas, impidiendo la continuidad del ataque.

La consulta realizada mediante `fail2ban-client status sshd` permitió verificar la correcta aplicación de la política de bloqueo.

Ver Figura 45.

Verificación del Bloqueo

Como etapa final del experimento se intentó establecer nuevamente una conexión SSH desde la máquina atacante hacia el servidor Ubuntu.

El intento de conexión fue rechazado automáticamente por el servidor debido a que la dirección IP atacante permanecía incluida en la lista de bloqueo administrada por Fail2Ban.

Esta prueba permitió validar experimentalmente que la respuesta automática implementada durante el laboratorio no solo detecta el ataque, sino que además impide efectivamente la continuidad de los intentos de acceso no autorizado.

Figura 46.

Correlación de Eventos

A partir del análisis de las evidencias obtenidas fue posible reconstruir la secuencia completa de eventos producidos durante el experimento.

Inicialmente, la herramienta Hydra ejecutó múltiples intentos automatizados de autenticación contra el servicio OpenSSH del servidor Ubuntu.

Cada intento fue procesado por el subsistema PAM, responsable de validar las credenciales suministradas, registrando posteriormente el resultado de la autenticación en los registros administrados por `systemd-journal`.

Fail2Ban monitoreó continuamente dichos registros, identificando la repetición de eventos de autenticación fallida provenientes de una misma dirección IP.

Una vez alcanzado el umbral configurado para el servicio SSH, el mecanismo de protección ejecutó automáticamente la acción de bloqueo correspondiente, incorporando la dirección IP atacante a la lista de direcciones bloqueadas del sistema.

Finalmente, al intentar establecer una nueva conexión SSH desde la máquina Kali Linux, el servidor rechazó la solicitud debido a la política de bloqueo aplicada automáticamente, demostrando el correcto funcionamiento del sistema automatizado de detección y respuesta implementado durante el laboratorio.

Conclusiones

El presente Trabajo Práctico Integrador permitió diseñar, implementar y validar un laboratorio de seguridad informática orientado a la detección y respuesta automática frente a intentos de fuerza bruta sobre un servicio SSH en un entorno Linux.

A lo largo del desarrollo se implementó una arquitectura compuesta por un servidor Ubuntu y una máquina atacante Kali Linux, integrando herramientas ampliamente utilizadas en el ámbito profesional como OpenSSH, Hydra y Fail2Ban.

Los resultados obtenidos durante los escenarios experimentales demostraron que el sistema fue capaz de registrar los intentos de autenticación fallida, identificar el patrón de ataque y aplicar automáticamente una política de bloqueo sobre la dirección IP atacante, mitigando de manera efectiva el intento de acceso no autorizado.

Asimismo, el proyecto permitió aplicar en un entorno práctico diversos conceptos abordados durante la asignatura Arquitectura de Sistemas Operativos, entre ellos la administración de servicios mediante *systemd*, los mecanismos de autenticación en Linux, el funcionamiento de los registros del sistema, la interacción entre procesos y la automatización de respuestas ante eventos de seguridad.

En conclusión, los objetivos planteados al inicio del trabajo fueron alcanzados satisfactoriamente, obteniéndose un laboratorio funcional, reproducible y documentado que evidencia la importancia de implementar mecanismos de monitoreo y protección para fortalecer la seguridad de los sistemas operativos modernos.

Bibliografía

- Canonical Ltd. *Ubuntu Server Documentation*.
- The OpenSSH Project. *OpenSSH Manual Pages*.
- Fail2Ban Community. *Fail2Ban Documentation*.
- THC-Hydra Project. *Hydra User Guide*.
- Silberschatz, A., Galvin, P. B. y Gagne, G. *Operating System Concepts*. Wiley.
- Stallings, W. *Operating Systems: Internals and Design Principles*. Pearson.